

Information Theory and Inference

MCMC BENCHMARKING IN COMPLEX
DISTRIBUTIONS
&
APPLICATION TO A REAL INFERENCE
PROBLEM: FISH TRACKING

Biliato Pietro - Salcido Ana Sofia - Waruszynski Sebastian

Introduction - MCMC

- MCMC methods objective is obtain **samples from a target distribution**. In Bayesian framework, the target distribution is referred to as the 'posterior', and can be calculated as:

$$\pi(\theta) = p(\theta|data) = \frac{p(data|\theta)p(\theta)}{Z(data)}$$

- To do this, the computation of integrals of the kind $\pi(f) = \int_{\mathcal{X}} f(x)\pi dx$ is needed. In complex distributions these integrals are intractable which lead to **marginalization** problems that become impossible to solve analytically
- To solve this we recur to Markov Chain Monte Carlo (MCMC) methods to approximate this marginalization and obtain the posterior distribution using the **ergodic average**

$$\pi(\hat{f})_m = \frac{1}{m+1} \sum_{n=0}^m f(\chi_n)$$

MCMC METHODS

Random-Walk Metropolis Hasting

- The posterior is sampled using a symmetric random walk proposal combined with a Metropolis acceptance rate
- Algorithm:

Let $\theta^i \in R^d$ denote the state of the chain at iteration i . The RWMH algorithm proceeds as follows:

1. Generate a proposal

$$\theta^* = \theta^i + c\Sigma^{1/2}W,$$

where c is a scalar scaling parameter, Σ is a positive definite proposal covariance matrix chosen by the user, and $W \sim \mathcal{N}(0, I_d)$.

2. Compute the acceptance probability

$$\alpha(\theta^i, \theta^*) = \min \left\{ 1, \frac{\pi(\theta^*)}{\pi(\theta^i)} \right\},$$

where $\pi(\theta)$ denotes the target posterior density.

3. Set

$$\theta^{i+1} = \begin{cases} \theta^* & \text{with probability } \alpha(\theta^i, \theta^*), \\ \theta^i & \text{otherwise.} \end{cases}$$

Hamiltonian Monte Carlo - NUTS

- It uses a gradient-driven algorithm to propose distance states with high acceptance probability
- 2 parameters have however to be tuned:
 - Step size, ϵ
 - Number of Leapfrog steps, L
- No-U-Turn Sampler eliminates the need to tune L

1. Sample momentum $r \sim \mathcal{N}(0, I)$.

2. Sample slice variable

$$u \sim \text{Uniform} \left(0, \exp \left(L(\theta^t) - \frac{1}{2} r^\top r \right) \right).$$

3. Recursively build the trajectory sets $\mathcal{B}$ and $\mathcal{C}$ using the leapfrog integrator until a U-turn condition is met.

4. Sample the next state (θ^{t+1}, r) uniformly from $\mathcal{C}$.

Parallel Tempering

- It uses tempered distributions, which have densities proportional to π^β for some $\beta \in (0, 1)$.
- PT works with $X_n = (X_n^1, \dots, X_n^L)$ Markov Chains
- The joint distribution is

$$\Pi_\beta(x^1, \dots, x^L) \propto \prod_{l=1}^L \pi(x^l)^{\beta^l}$$

- The algorithm consist on 2 steps
 - **Propagation:** Each component evolves independently according to a π^{β^l} -invariant kernel. The joint transition kernel is

$$M_\beta(x, y) = \prod_{l=1}^L M_{\beta^l}(x^{(l)}, y^{(l)}),$$

where M_{β^l} is reversible with respect to π^{β^l} .

- **Swap:** Choose uniformly at random an index $l \in \{1, \dots, L-1\}$ and propose to swap the states $x^{(l)}$ and $x^{(l+1)}$. The proposal is accepted with Metropolis probability

$$\alpha = \min \left\{ 1, \frac{\pi(x^{(l+1)})^{\beta^l} \pi(x^{(l)})^{\beta^{l+1}}}{\pi(x^{(l)})^{\beta^l} \pi(x^{(l+1)})^{\beta^{l+1}}} \right\}.$$

More on Parallel Tempering: Pigeons.jl

- Pigeons.jl is a Julia package to approximate challenging posterior distributions.
- Able to do parallelization using MPI framework
- Core algorithm
 - Non-reversible Parallel Tempering:
 - Variational Parallel Tempering
 - AutoMALA
- Explorers: Is the algorithm responsible for generating the MCMC updates
 - SliceSampler: It consists of sampling uniformly from the region under the graph of the target density function
 - AutoMALA: Is based on the approximation of the overdamped Langevin dynamics followed by a Metropolis-Hasting correction and is gradient-drive

More on Parallel Tempering: Pigeons.jl

- **Annealing Path:**

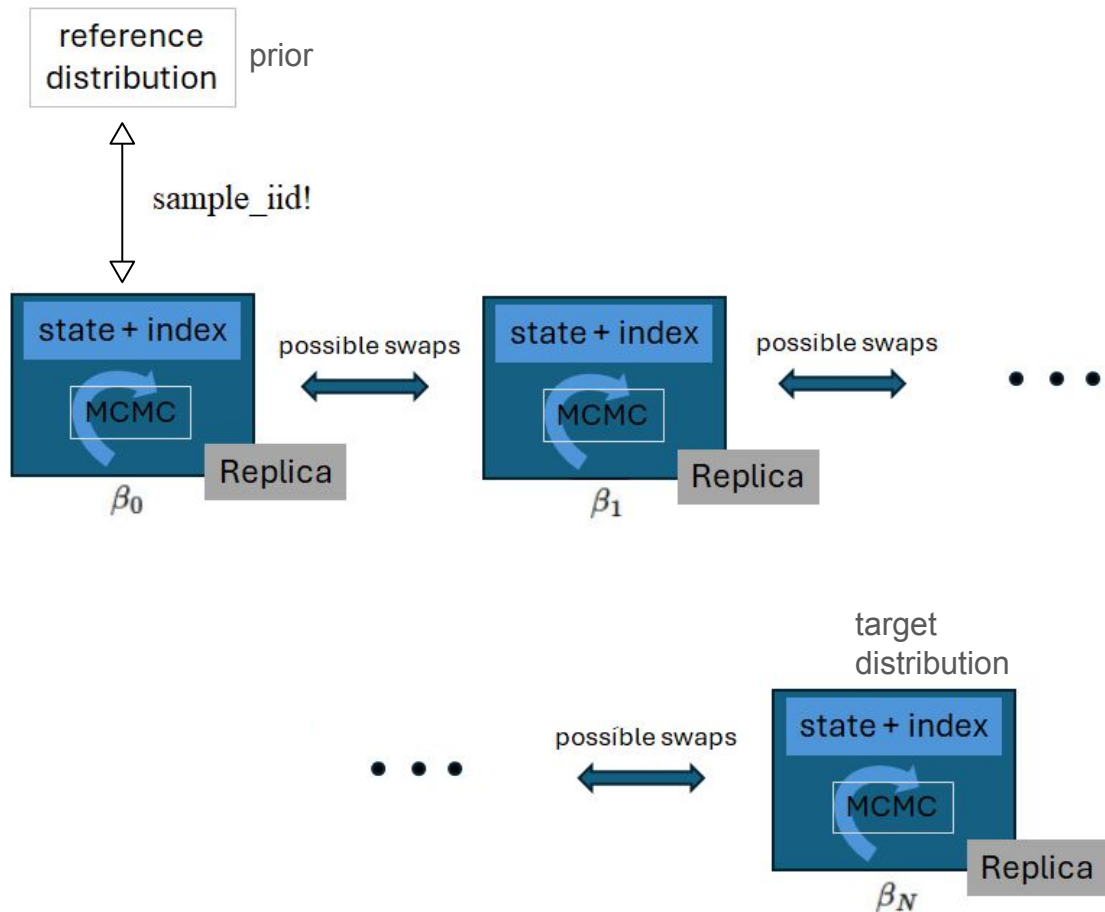
PT constructs a sequence of distributions $\pi_1, \dots, \pi_N$.

- π_1 is the reference distribution (easy to sample, e.g., the prior).
- π_N is the target distribution (the posterior π).

- **Local exploration and communication (two-phase cycle):**

PT works with *replicas* that store a state variable and a chain integer. At the beginning N replicas are created, one for each annealing path. For a given replica, if a chain i is in the state $\mathbf{x}$ this means that π_i sample is currently in location $\mathbf{x}$

- Exploration phase: Each replica state is modified by a MCMC move targeting π_i
- Communication phase: PT propose swap between replicas; this facilitates global mixing across the annealing path
 - Swap the chains index fields: The current distribution π_i is the targeted or
 - Swap the states fields: The current location $\mathbf{x}$ in the parameter space



Algorithm 1 Distributed PT on machine j (one replica per machine)

Require: Initial state x_0 , sequence of distributions $\{\pi_i\}_{i=1}^N$, number of iterations T , machine number j

```

1: chain  $\leftarrow j$   $\triangleright$  current chain number
2: dist_array[j]  $\leftarrow j$   $\triangleright$  chain  $j$  is on machine  $j$ 
3: for  $t$  in  $1, 2, \dots, T$  do
4:   if  $t$  is even then  $\triangleright$  choose between even or odd swap
5:      $P \leftarrow \{i : 1 \leq i < N, i \text{ is even}\}$ 
6:   else
7:      $P \leftarrow \{i : 1 \leq i < N, i \text{ is odd}\}$ 
8:   end if
9:    $x_t \leftarrow \text{local\_exploration}(\pi_{\text{chain}}, x_{t-1})$ 
10:  partner  $\leftarrow$  nothing
11:  if chain  $\in P$  then
12:    partner  $\leftarrow$  chain + 1
13:  else if chain - 1  $\in P$  then
14:    partner  $\leftarrow$  chain - 1
15:  end if
16:  if partner  $\neq$  nothing and swap is accepted then
17:    to_machine  $\leftarrow$  permuted_get(dist_array, partner)
     $\triangleright$  retrieve dist_array[partner]
18:    send_tag  $\leftarrow$  tag( $t$ , chain,  $j$ )
19:    send(chain, to_machine, send_tag)
20:    receive_tag  $\leftarrow$  tag( $t$ , chain, to_machine)
21:    receive!(chain, to_machine, receive_tag)
22:    waitall(requests)  $\triangleright$  wait for MPI requests for this
    pair of machines to complete
23:    permuted_set!(dist_array, chain,  $j$ )  $\triangleright$  set value
    of dist_array[chain] to to_machine
24:  end if
25:   $c_t \leftarrow$  chain
26: end for
27: return  $\{(x_t, c_t)\}_{t=1}^T$ 

```

**Non-Reversible
PT**

BENCHMARKING METRICS

Potential Scale Reduction Factor $\hat{R}$

- Convergence diagnostic that assesses whether multiple Markov chains have reached their stationary distribution.
- The statistic measures the ratio between the estimated marginal posterior variance and the average within-chain variance

$$B = \frac{N}{M-1} \sum_{m=1}^M \left(\bar{\theta}_m^* - \bar{\theta}_*^* \right)^2, \quad W = \frac{1}{M} \sum_{m=1}^M s_m^2, \quad \widehat{\text{Var}}(\theta | y) = \frac{N-1}{N} W + \frac{1}{N} B, \quad \hat{R} = \sqrt{\frac{\widehat{\text{Var}}(\theta | y)}{W}}.$$

- **Threshold:** the closer R is to 1, the better the convergence.

Effective Sampling Size (/ wall time)

- MCMC chains are correlated (not i.i.d)
- ESS represents the number of *independent* samples that would provide the same precision as the $M \times N$ MCMC samples.

$$N_{\text{eff}} = \frac{M \cdot N}{\hat{\tau}} \qquad \hat{\tau} = -1 + 2 \sum_{t'=0}^m \hat{P}_{t'}. \qquad \hat{P}_{t'} = \hat{\rho}_{2t'} + \hat{\rho}_{2t'+1}$$

- When $N_{\text{ess}} \approx M \cdot N$, the autocorrelation is small and the sampling efficiency is high.
- **ESS/time**: Measures how many i.i.d., equivalent samples, are obtained per unit of wall time. It is the ultimate **performance metric**, accounting for both statistical efficiency and computational cost

Parallel Tempering: Index Process

- Communication and information flow between the target distribution, the intermediate distributions, and the reference distribution across the annealing path
- It consist in 2 modes:
 - DEO (Deterministic Even-Odd): The proposal indicator is deterministic (Non-Reversible PT)
 - SEO (Stochastic Even-Odd): The communication behave like a random walk in the temperature index

$$(I_n^j, \varepsilon_n^j) \in \{0, \dots, N\} \times \{-1, 1\}.$$

$$S_n^{(i,i+1)} = P_n^{(i,i+1)} A_n^{(i,i+1)}, \quad \text{Realized swaps}$$

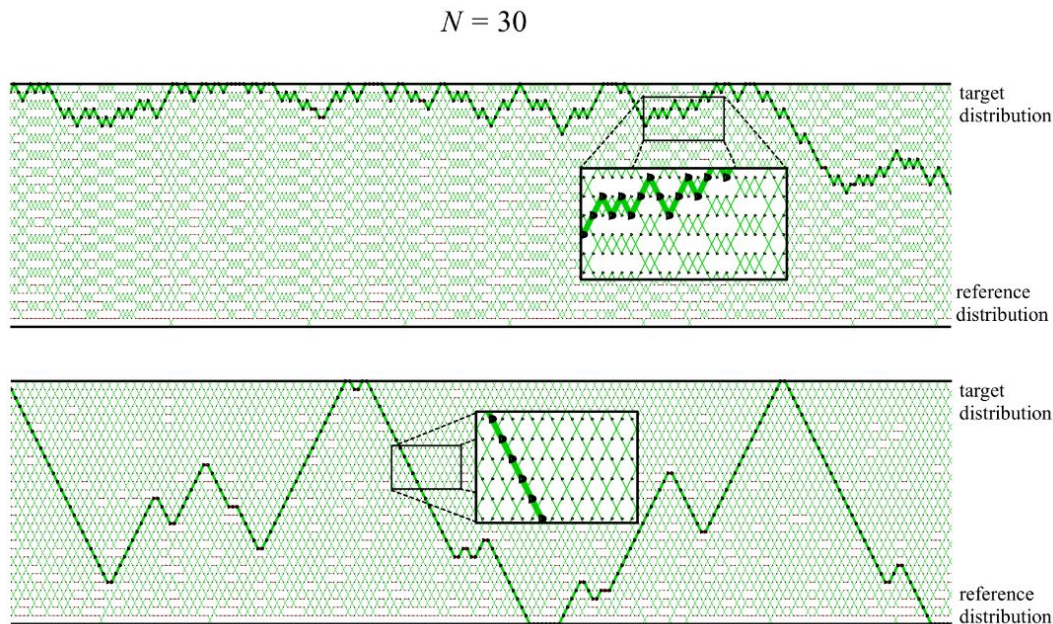
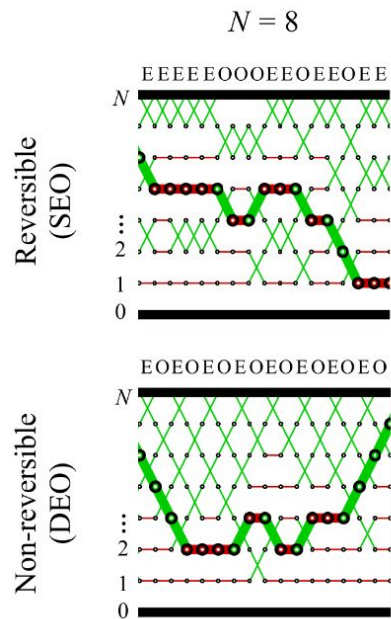
$\downarrow$
 Swap
indicator

$\downarrow$
 Acceptance
indicator

$$I_{n+1}^j = \begin{cases} I_n^j + \varepsilon_n^j & \text{if } S_n^{(I_n^j, I_n^j + \varepsilon_n^j)} = 1, \\ I_n^j & \text{otherwise,} \end{cases}$$

$$\varepsilon_{n+1}^j = \begin{cases} 1 & \text{if } P_n^{(I_{n+1}^j, I_{n+1}^j + 1)} = 1, \\ -1 & \text{otherwise.} \end{cases}$$

Parallel Tempering: Index Process



Parallel Tempering: Global Communication Barrier

- Measures the cumulative difficulty of information flow between the reference distribution π_0 and the target distribution π
- The local communication barrier is defined as

$$\lambda(\beta) = \lim_{\delta \rightarrow 0} \frac{r(\beta, \beta + \delta)}{|\delta|}$$

- The global communication barrier is then

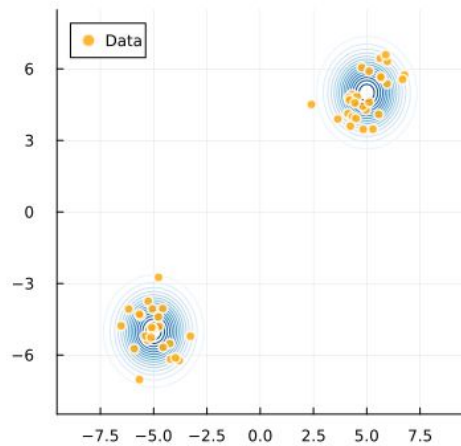
$$\Lambda = \int_0^1 \lambda(\beta) d\beta.$$

- The optimal number of temperature levels satisfies

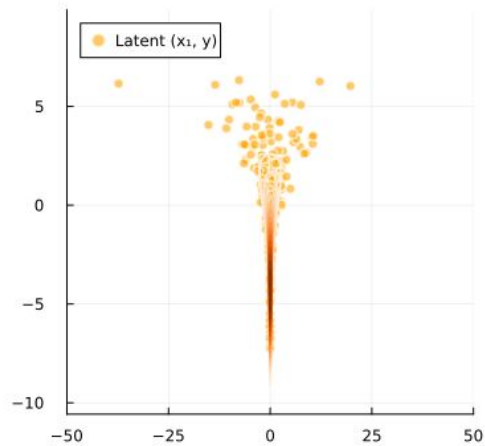
$$N + 1 \approx 2\Lambda + 1, \quad N \approx 2\Lambda.$$

DISTRIBUTIONS

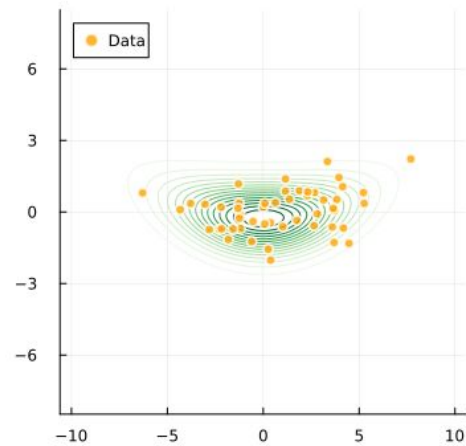
Gaussian Mixture



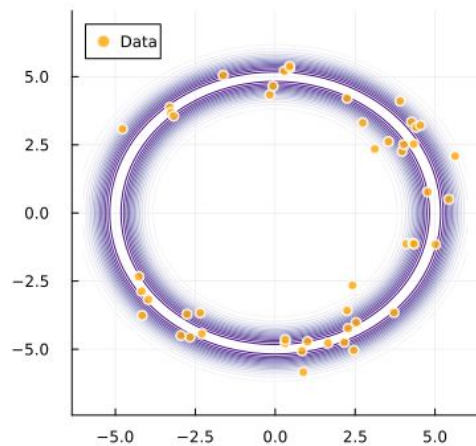
Neal's Funnel (x_1 vs y)



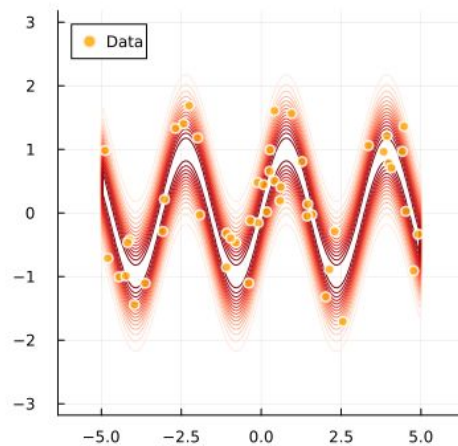
Banana



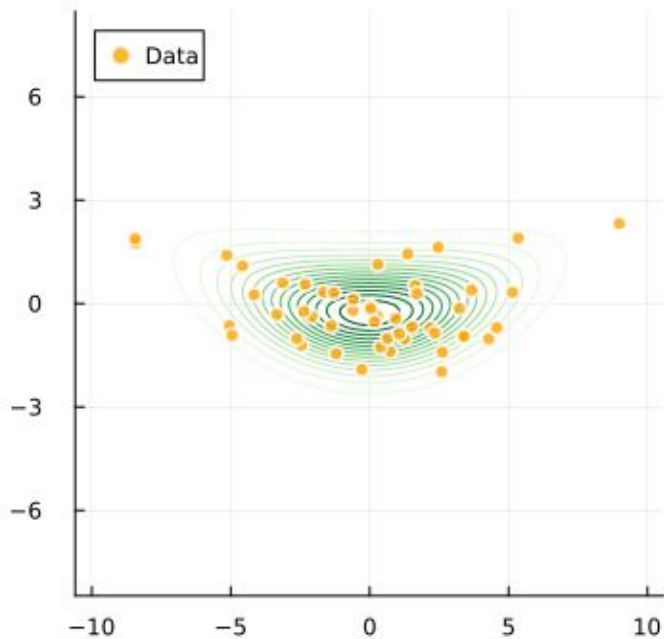
Donut



Squiggle



Banana



Likelihood:

$$x_{i,1} \sim \mathcal{N}(\mu_1, \sigma_1^2)$$

$$x_{i,2} \sim \mathcal{N}(b(x_{i,1}^2 - 9.0) + \mu_2, \sigma_2^2)$$

Priors:

$$b \sim \mathcal{U}(0.0, 0.1)$$

$$\mu_1, \mu_2 \sim \mathcal{N}(0.0, 5^2)$$

$$\sigma_1, \sigma_2 \sim \mathcal{U}(0.1, 5.0)$$

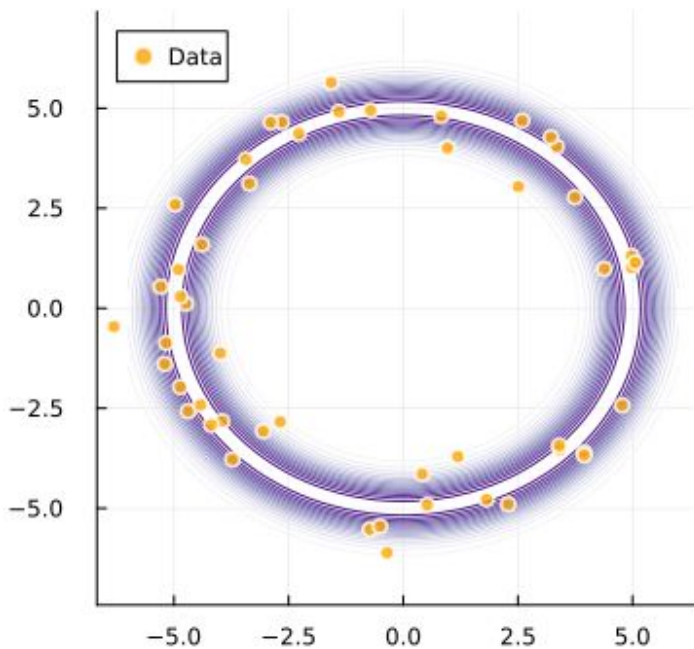
True values:

$$b = 0.03$$

$$\mu_1 = 0.0, \mu_2 = 0.0$$

$$\sigma_1 = 3.0, \sigma_2 = 1.0$$

Donut



Likelihood:

$$r_i \sim \mathcal{N}(R, \sigma^2)$$

Priors:

$$R \sim \mathcal{U}(1.0, 10.0)$$

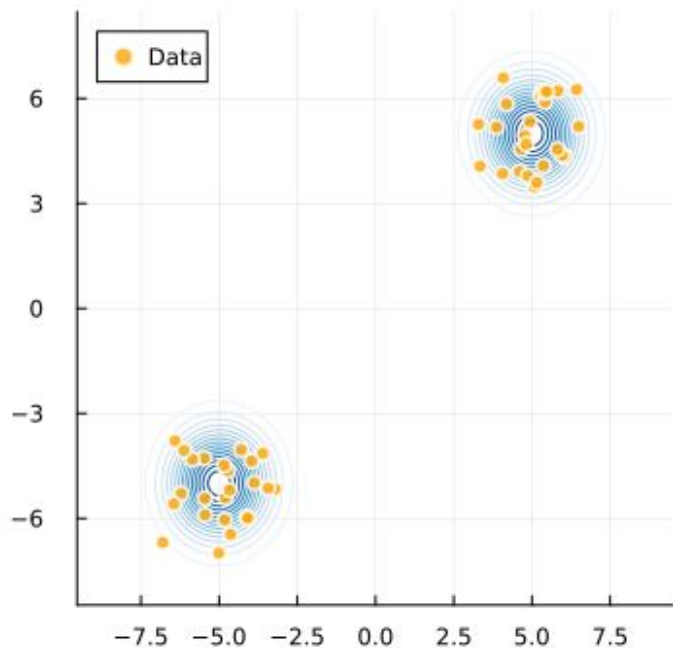
$$\sigma \sim \mathcal{U}(0.1, 2.0)$$

True values:

$$R = 5.0$$

$$\sigma = 0.5$$

Gaussian Mixture



Likelihood:

$$\mathbf{x}_i \sim 0.5 \mathcal{N}(\mu_1, I) + 0.5 \mathcal{N}(\mu_2, I)$$

Priors:

$$\mu_1 \sim \mathcal{N}([-2.0, -2.0]^\top, 5.0I)$$

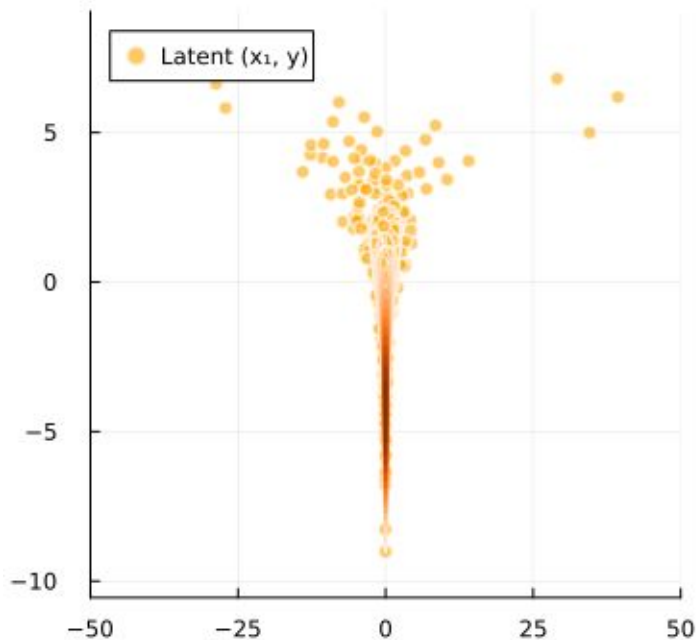
$$\mu_2 \sim \mathcal{N}([2.0, 2.0]^\top, 5.0I)$$

True values:

$$\mu_1 = [-5.0, -5.0]^\top$$

$$\mu_2 = [5.0, 5.0]^\top$$

Neal's Funnel (x_1 vs y)



Likelihood:

$$d_i \sim \mathcal{N}(\mathbf{x}, I_8)$$

Priors:

$$y \sim \mathcal{N}(0, 3^2)$$

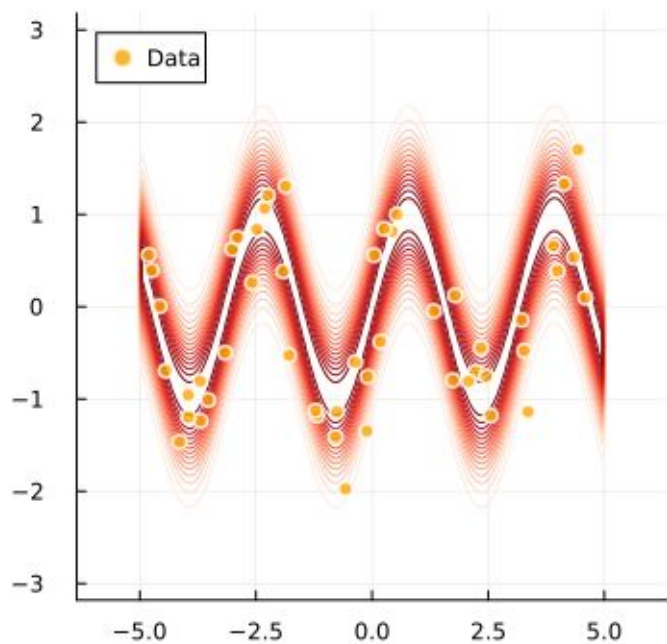
$$\mathbf{x} \sim \mathcal{N}(\mathbf{0}, \exp(y/2)^2 I_7)$$

True values:

$$y_t \sim \mathcal{N}(0, 3^2)$$

$$\mathbf{x}_t \sim \mathcal{N}(\mathbf{0}, \exp(y/2)^2 I_7)$$

Squiggle



Likelihood:

$$x_{i,1} \sim \mathcal{U}(-5.0, 5.0)$$

$$x_{i,2} \sim \mathcal{N}(\sin(ax_{i,1}), \sigma^2)$$

Priors:

$$a \sim \mathcal{U}(0.5, 5.0)$$

$$\sigma \sim \mathcal{U}(0.1, 2.0)$$

True values:

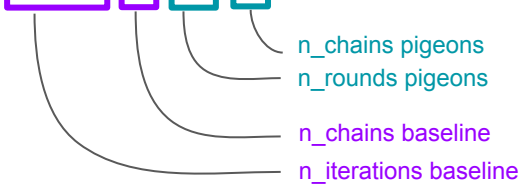
$$a = 2.0$$

$$\sigma = 0.5$$

ANALYSIS & RESULTS

ANALYSIS

Tested configurations:

- 9000_x-13_x, $x=[3,4,6]$

 - n_chains pigeons
 - n_rounds pigeons
 - n_chains baseline
 - $n_iterations$ baseline
- 32800_x-15_x, $x=[3,4,6]$
- 65500_4-16_4
- 10000_70-17_4

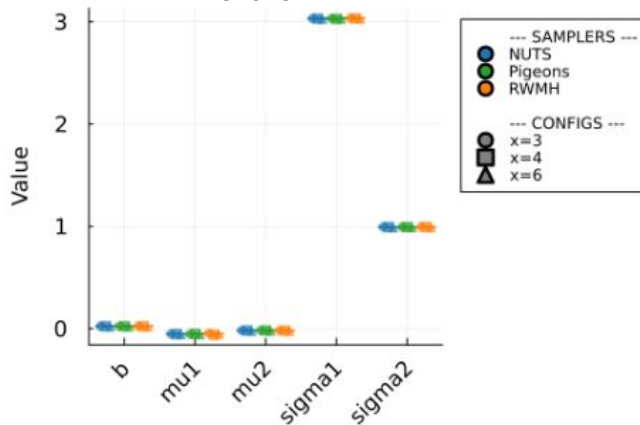
In Pigeons, the number *iterations* per chain (scans) doubles each *round*: the final tempering round performs 2^{n_rounds} *iterations*.

The baseline samplers' *iteration* count were matched to 2^{n_rounds} value, for a 1:1 comparison of the local exploration budget per chain between the algorithms, knowing however that the samples only come from the “cold” chain.

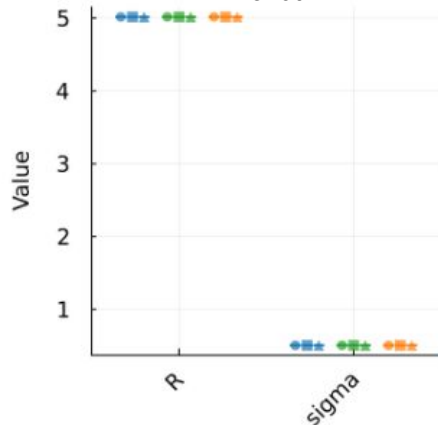
350 datapoints were used (random seed: 44), to reduce the influence of the prior; a *thinning* factor of 10 was used for the baseline samplers, as well as a *burn-in* set to 1000.

RESULTS - 9000_x-13_x - means $\pm$ MCSE

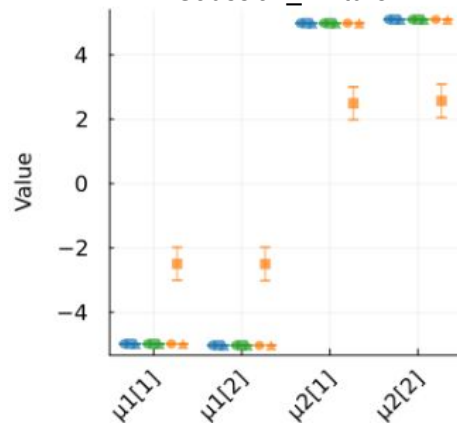
Banana



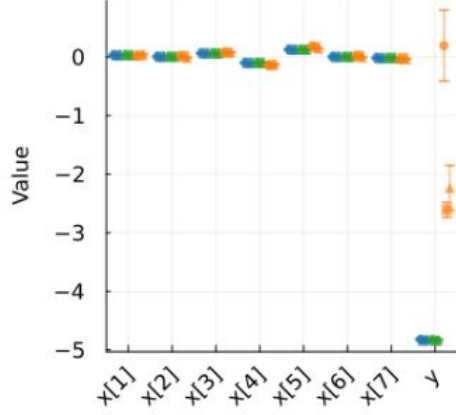
Donut



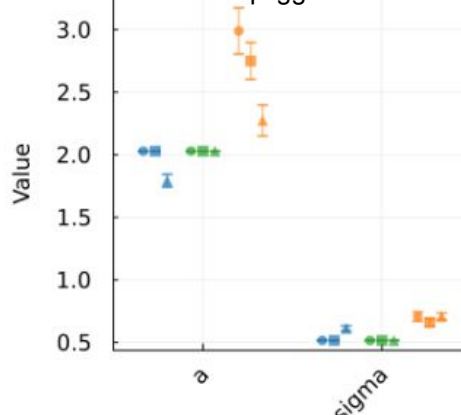
Gaussian_Mixture



Neal's Funnel



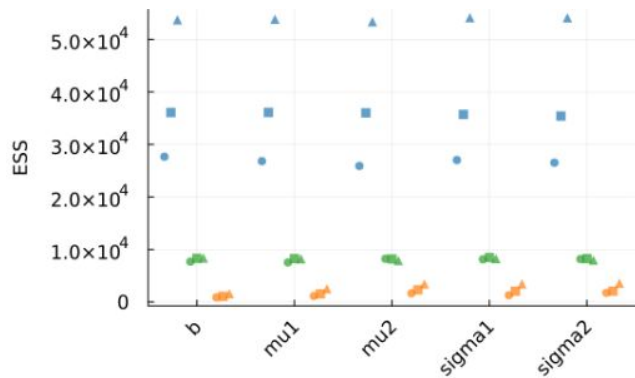
Squiggle



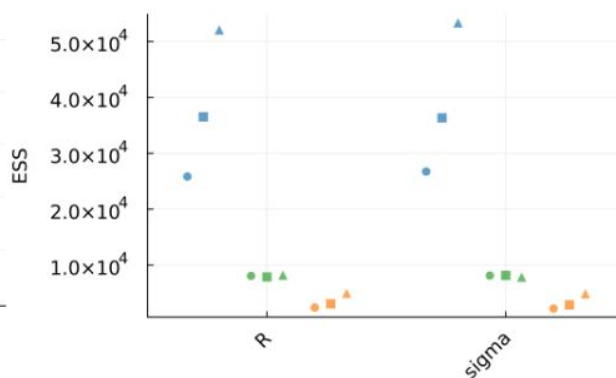
- $MCSE = \sqrt{\frac{\sigma^2}{N_{eff}}}$
- means instead of MAP were used because they better highlight some inconsistencies

RESULTS - 9000_x-13_x - ESS

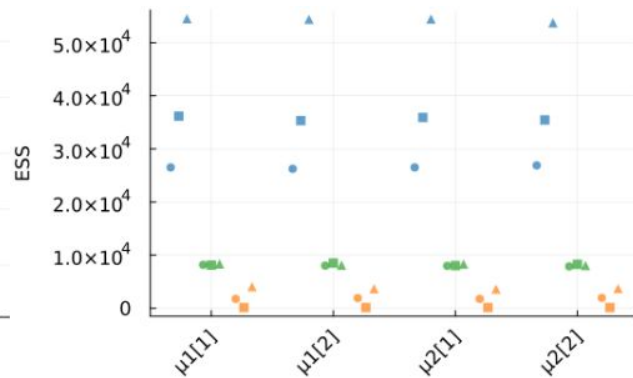
Banana



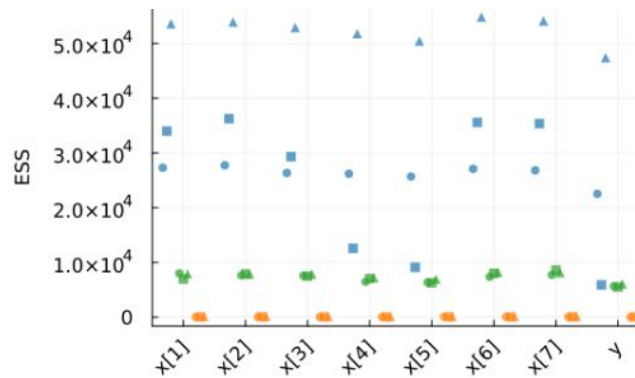
Donut



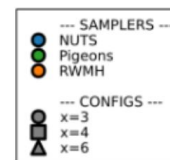
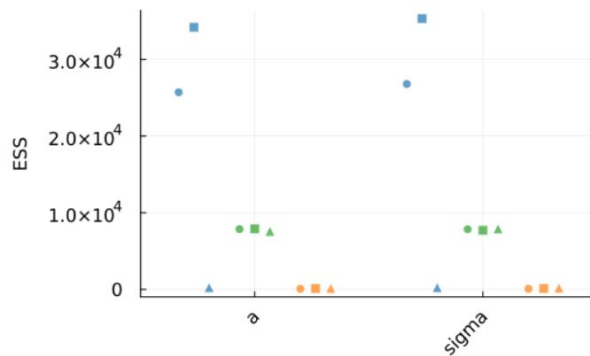
Gaussian_Mixture



Neal's Funnel

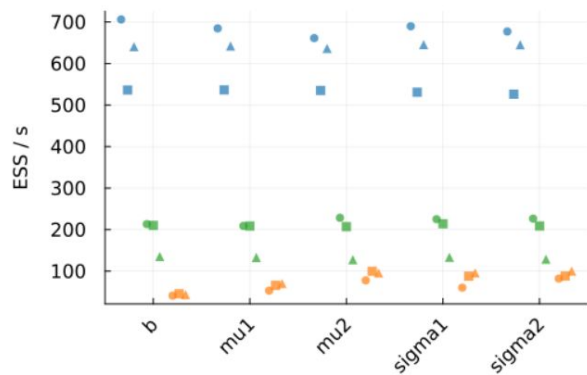


Squiggle

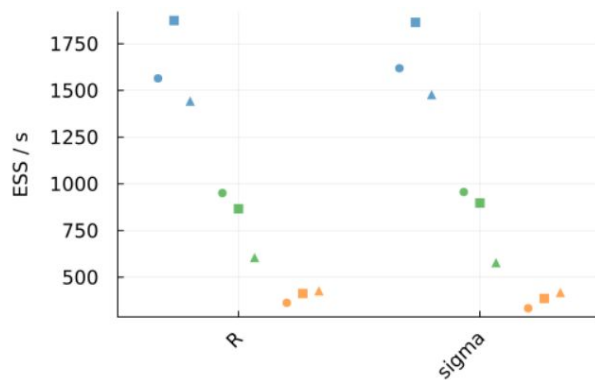


RESULTS - 9000_x-13_x - ESS/s

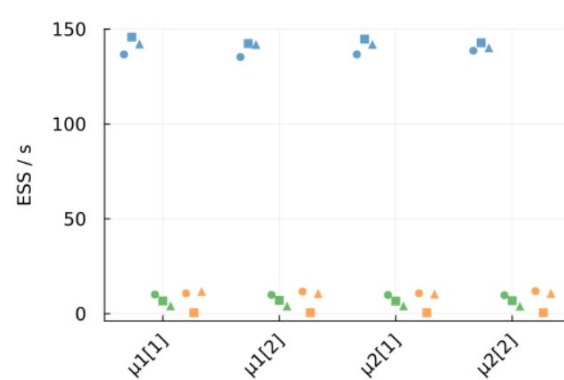
Banana



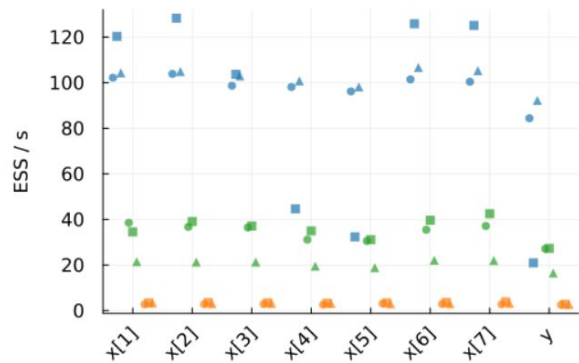
Donut



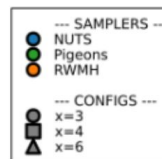
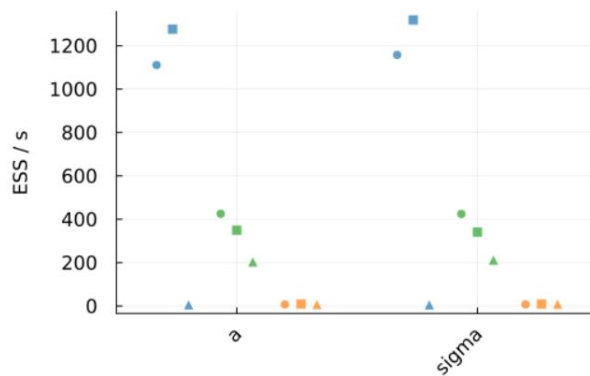
Gaussian_Mixture



Neal's Funnel

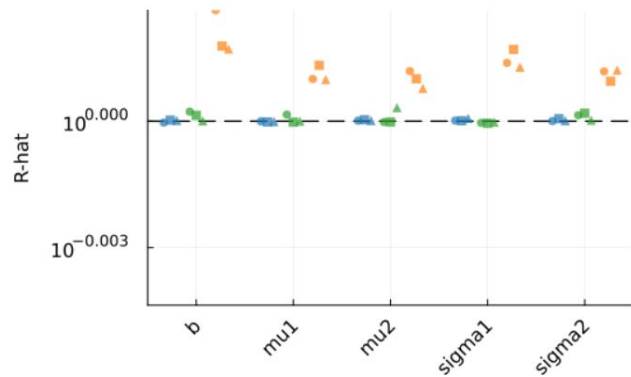


Squiggle

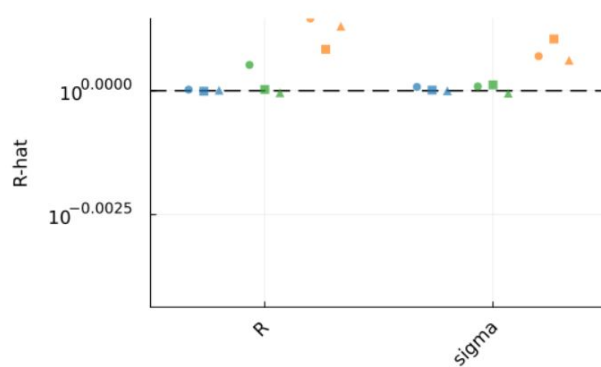


RESULTS - 9000_x-13_x - R_hat

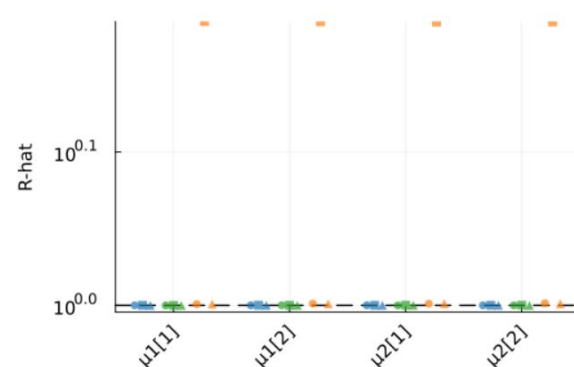
Banana



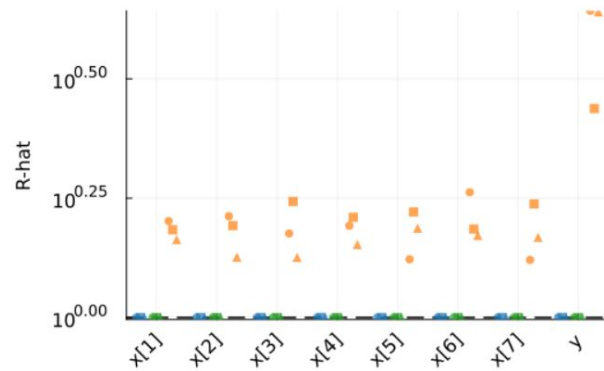
Donut



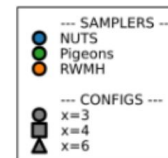
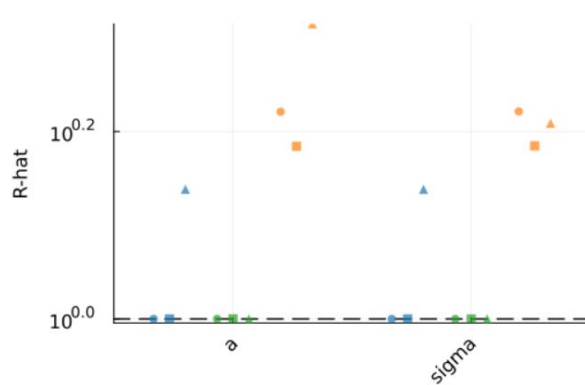
Gaussian_Mixture



Neal's Funnel



Squiggle

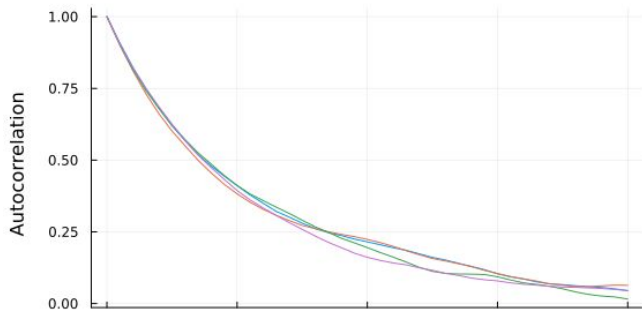


RESULTS - 9000_x-13_x - ACF

Typical decays: *Banana* - 9000_4-13_4

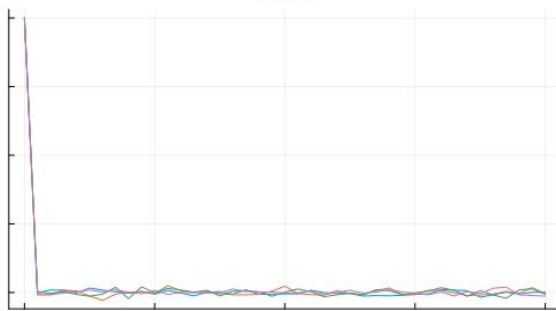
RWMH

mu1



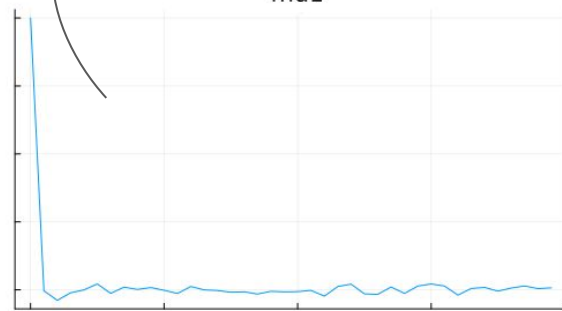
NUTS

mu1

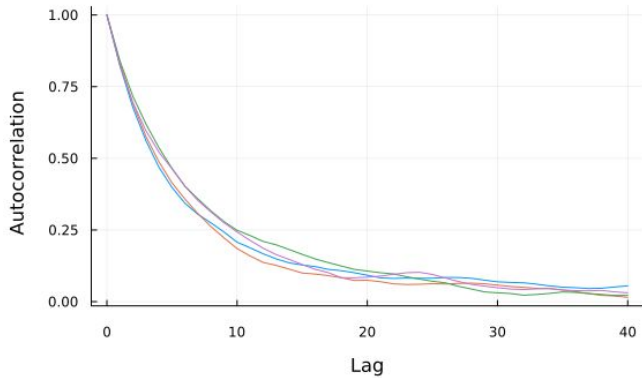


Pigeons

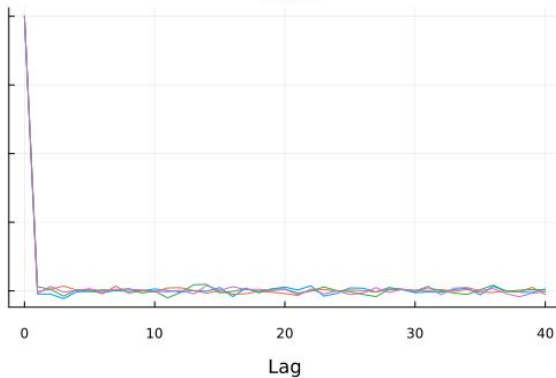
mu1



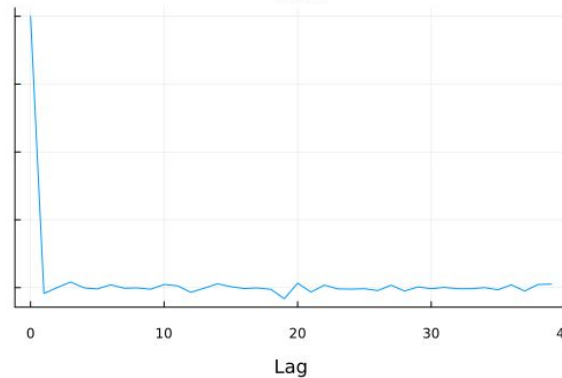
mu2



mu2

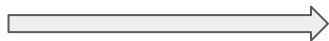
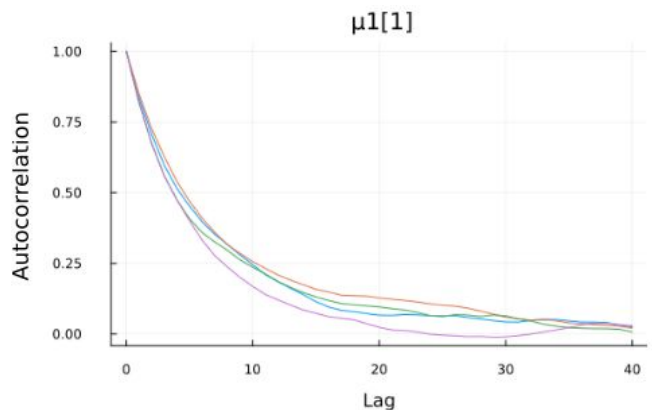


mu2

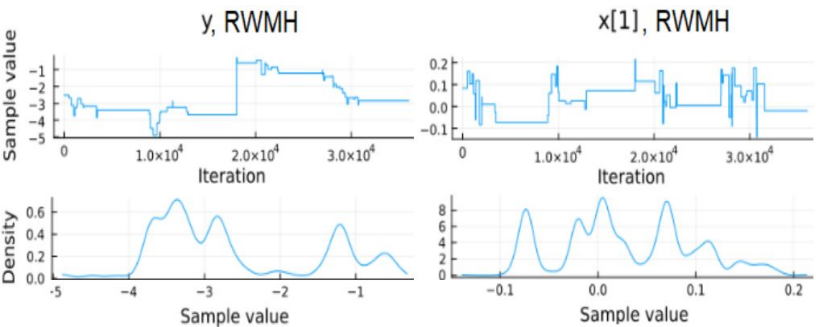
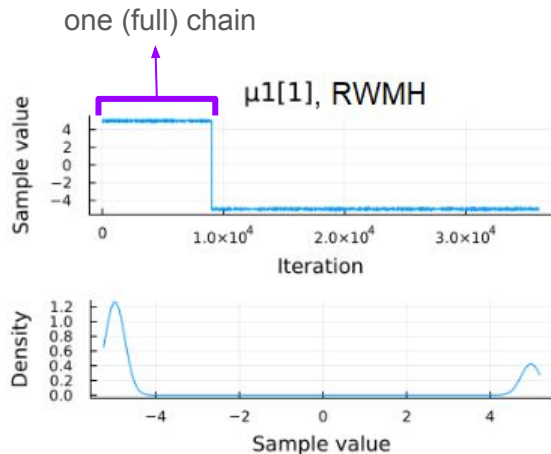


RESULTS - 9000_x-13_x - ACF

RWMH failure: Gaussian_Mixture - 9000_4-13_4

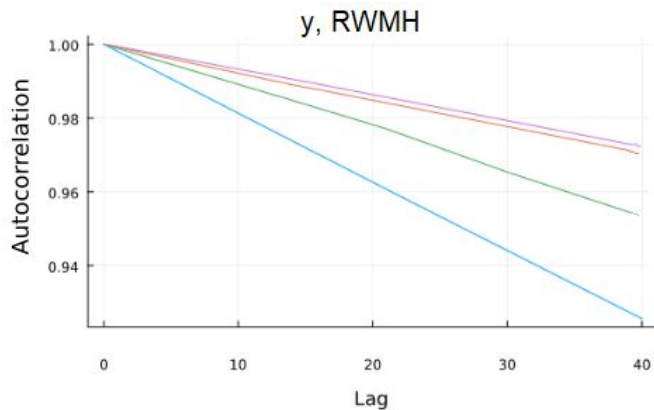


The issue doesn't seem to be related to the autocorrelation, and infact:



The ACF plots for all the parameters, for all x are like this one → the issue seems to be related to the autocorrelation, and infact:

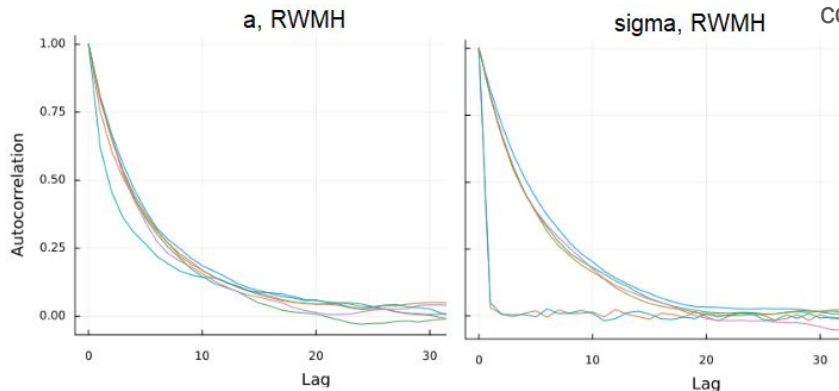
RWMH failure: Neal's Funnel - 9000_x-13_x



→ RWMH estimates in N.F. are NOT trustworthy

9000_x-13_x - ACF

RWMH failure: Squiggle - 9000_x-13_x



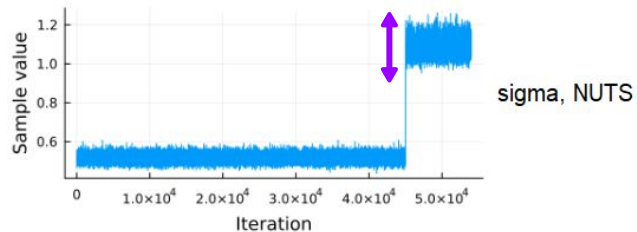
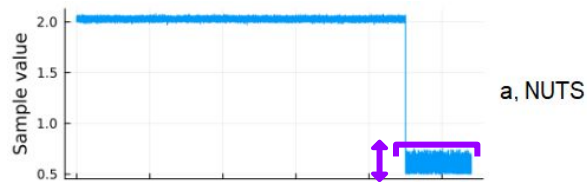
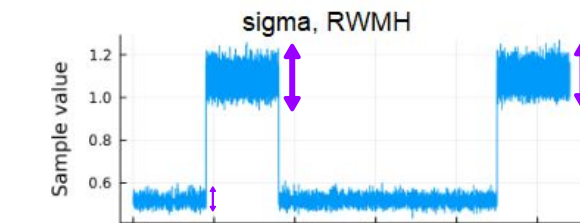
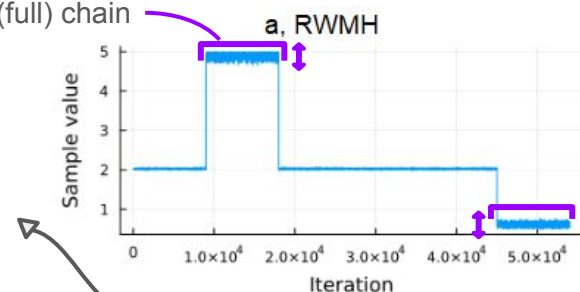
Note:

- the “illed” chains are the same across the parameters
- the different variability of the samples from the “wrong” regions, confirms they are indeed wrong.



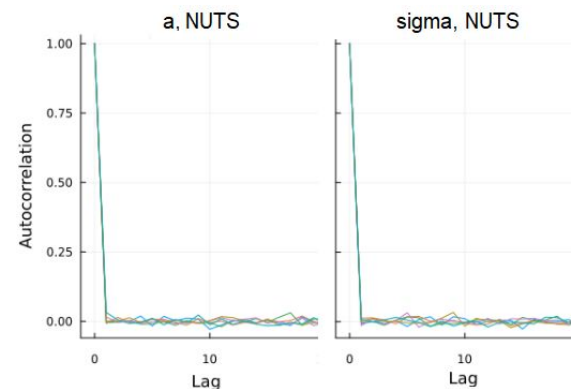
The ACF plots for *each* x are like this one → the issue does not seem to be related to the autocorrelation, and infact:

one (full) chain

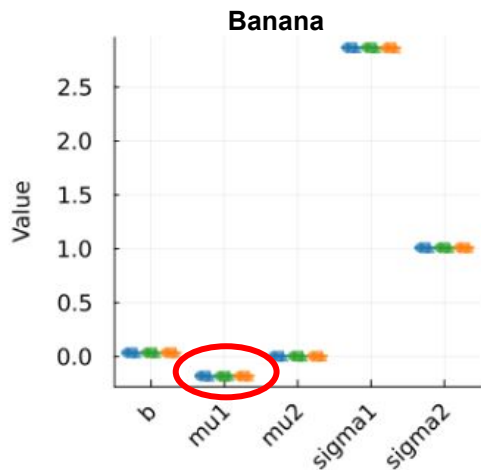


The issue is not related to the autocorrelation, infact:

NUTS failure: Squiggle - 9000_6-13_6

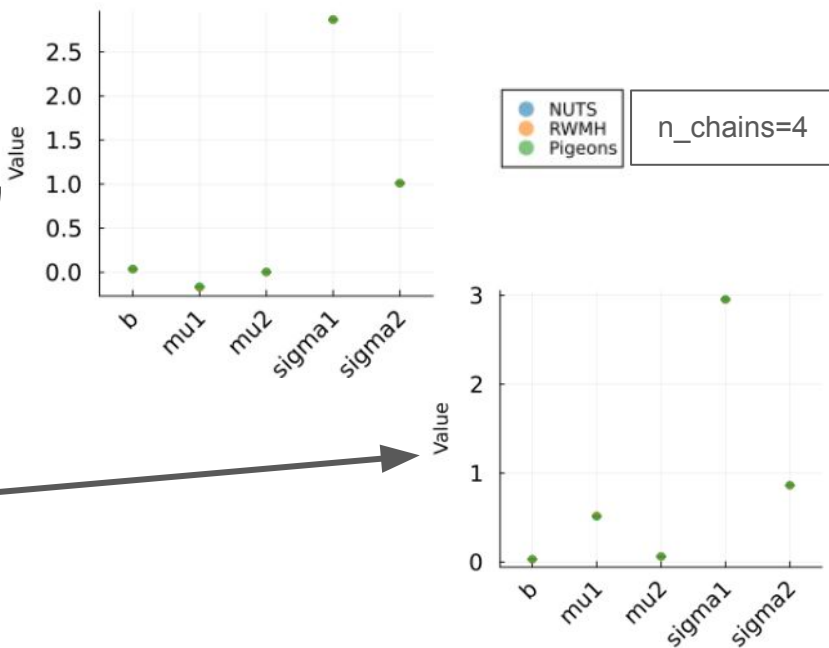


RESULTS - 9000_x-13_x - means $\pm$ MCSE - seed changed (42)

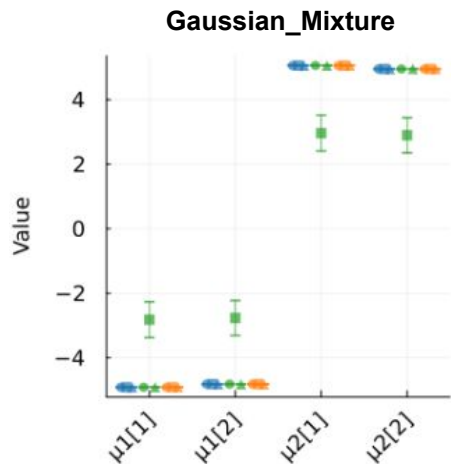


- Systematic error in the estimate of 'mu1', by all the samplers:
- the true posterior mean has shifted away from the generating 'mu1': sample mean: -0.2 ± 0.2 , estimate: -0.1818 ± 0.0007 (e.g. NUTS, 6 chains)

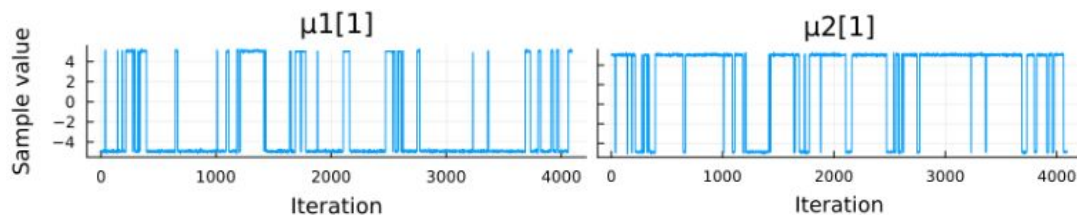
- Even with a more stringent prior on mu1 ($N(0,0.5)$) the estimates remain essentially identical (because the likelihood dominates the posterior)
- With fewer data points (50) & the stringent prior on mu1, the estimates *can* be even worse (if the sample mean is "far" from 0)



RESULTS - 9000_x-13_x - means $\pm$ MCSE - seed changed (42)



- RWMH is now producing accurate estimates $\rightarrow$ unstable algorithm.
- That of Pigeons is not a real failure:

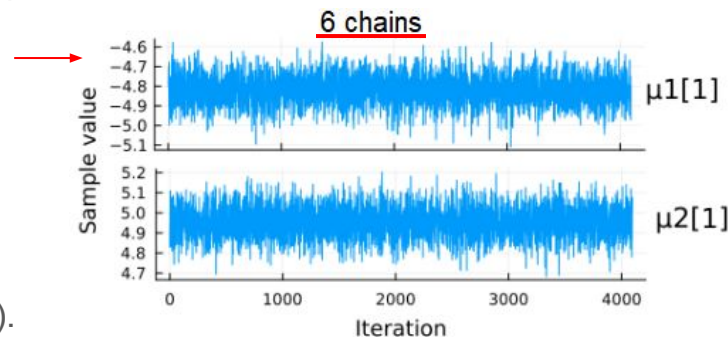


$\rightarrow$ the sampler is actively recognizing it is sampling values from the wrong region: remarkable result, PT is actually exploring the posterior.

- The cause of falling in the wrong region lies in using an improper number of chains:

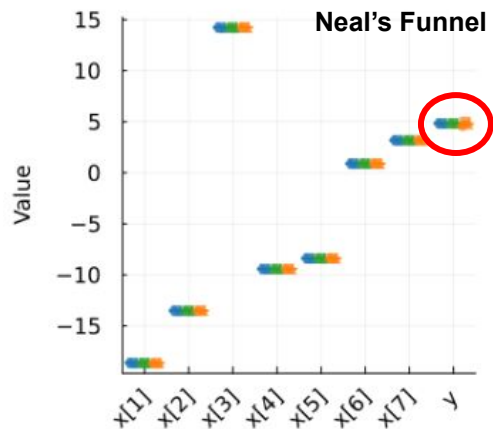
scans	restarts	λ
2	0	2
...		
2.05e+03	0	2.84
4.1e+03	0	2.86
8.19e+03	4	2.85

n_{chains} should be at least $2*3=6$;
 λ actually stabilizes at 5 $\rightarrow$ proper
 n_{chains} : 10

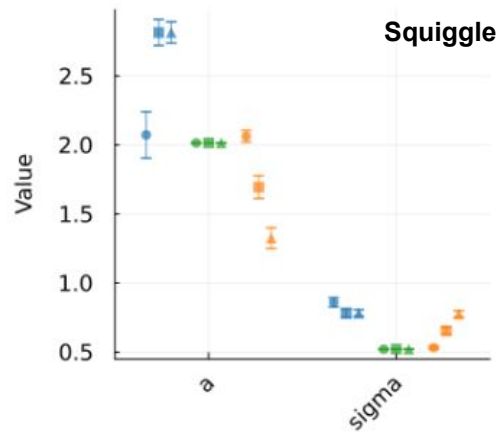
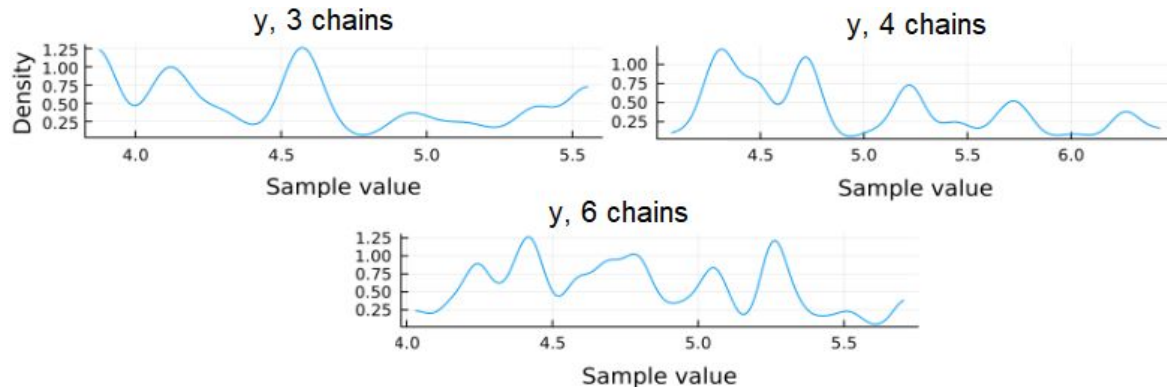


- The issue will also be solved by a higher n_{rounds} (see later).

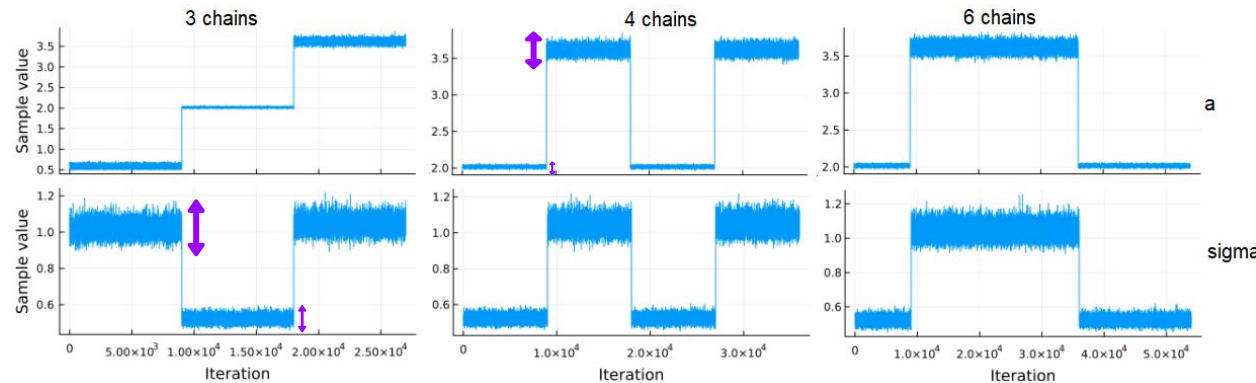
RESULTS - 9000_x-13_x - means $\pm$ MCSE - seed changed (42)



- RWMH is now producing accurate estimates $\rightarrow$ unstable algorithm.



- Total failure of NUTS



RESULTS - 9000_x-13_x - Summary

- RWMH looks structurally inadequate for distributions with high curvature due to its rigid (constant and isotropic) proposal mechanism;
- RWMH is highly sensitive to initial conditions for distributions with high curvature or disconnected modes;
- NUTS is highly efficient in simple geometries but demonstrated high sensitivity to initial conditions in complex topologies;
- PT shows actual exploration of the posterior, and if the temperature ladder is properly tuned (easy task), the algorithm has showed stable performances, accepting the computational overhead of thermodynamic synchronization.

To further assess these statements, longer runs have been performed.

RWMH should behave identically, NUTS may change, Pigeons should maintain stability and improve.

RESULTS - 32800_x-15_x - original seed (44)

- The failures arise in the same exact scenarios previously seen, and they are caused by the same phenomena (chains sampling from the wrong regions, RWMH not being able to explore Neal's Funnel topology);
- ESS, ESS/s and R_hat show the same patterns seen. For “non-failing” RWMH though, R_hat gets closer to 1;

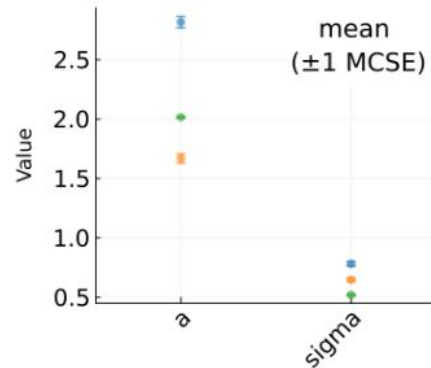
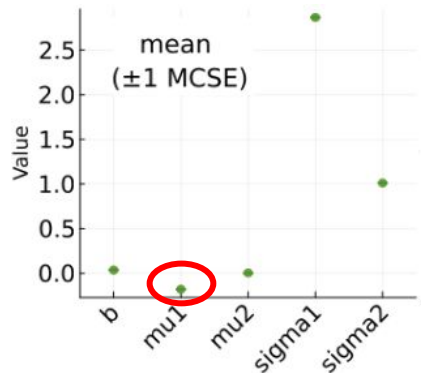
RESULTS - 65500_x-16_x - original seed (44)

- Nothing changes in the patterns of the means estimates with respect to the results obtained for ‘32800-x_15-x’;
- The ESS obviously grows but the patterns remain unchanged;
- The ESS/s remains basically constant between 32800 and 65500, even if ESS grows → the samplers are operating at a stable, linear rate of efficiency
- R-hat gets closer to 1 for the “non-failing” configurations of RWMH

The additional iterations
are not needed

RESULTS - 32800_4-15_4 - changed seed (42)

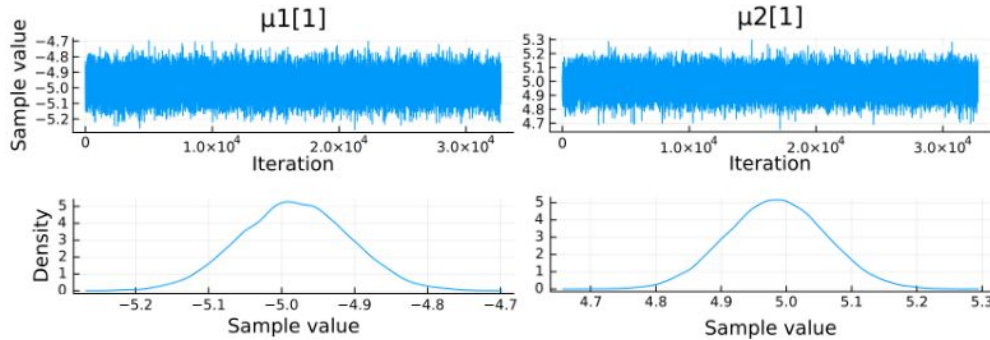
- 'mu1's in 'Banana' match the values obtained in the '9000' runs (seed 42) for all samplers:
- NUTS fails in 'Squiggle' when using 4 chains



- Pigeons manages to solve the "issue" encountered in 'Gaussian_Mixture' already using 4 chains: the higher *n_rounds* allows more 'restarts', thus a better performing exploration of the posterior:

scans	restarts	Λ
2	0	2
...		
8192	4	2.85
16384	11	2.85
32768	31	2.87

still not using the suggested *n_chains*, yet getting a satisfactory result



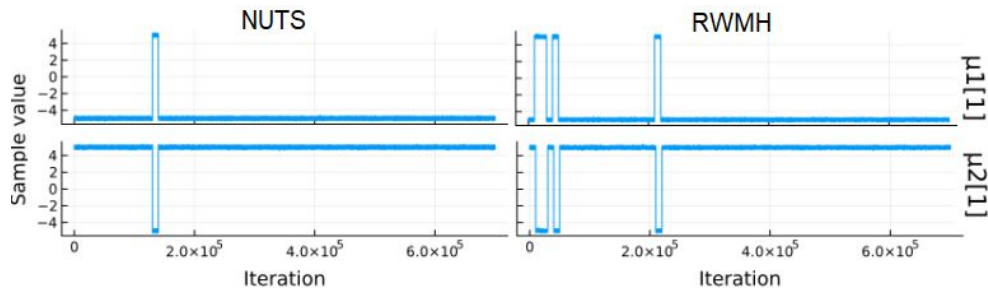
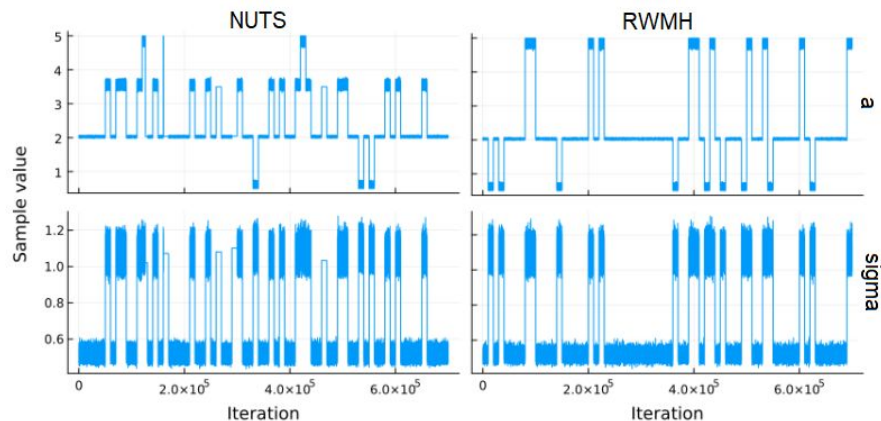
RESULTS - 32800_x-15_x, 65500-4_16-4 - Summary

- It has been shown that the algorithms have overcome transient warm-up phases and are actually sampling the typical set by 9000 iterations already, because:
- of the consistency of the means estimates across the samplers;
- of the constancy of ESS/s;
- of the stability of R_{hat} ;
- much longer runs do not affect the instability to initial conditions (in PT the issue is solved already by the number of chains).
- So running the chains longer purely reduces the Monte Carlo standard error (which is already small enough by 9000 iterations), and improves the exploration of RWMH, which however are not good enough reasons to wait 5 to 10 times more to get these results.

RESULTS - 10000_70-17_4

No direct equivalence to 70 chains can be carried out for Pigeons within any reasonable time on a single laptop; moreover, the results obtained so far do not support its usefulness

- Increasing the number of independent baseline chains did **not** solve complex geometries nor multimodality, instead it **exacerbated the fragility** of NUTS and RWMH:
- In the 'Squiggle' distribution, multiple chains sampled from spurious tails
- In the GMM, both NUTS and RWMH chains locked into the opposite, incorrect mode



- NUTS and RWMH are Markovian and cannot communicate across chains $\rightarrow$ increasing 'n_chains' simply increases the statistical probability of drawing a starting position that falls into a topological trap.

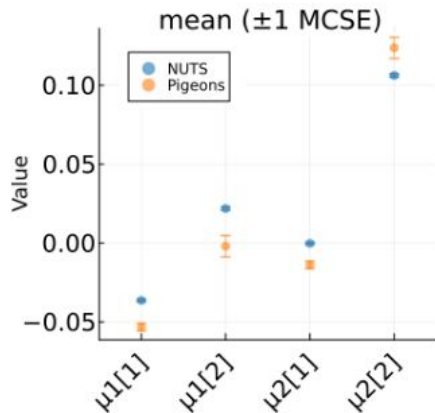
RESULTS - Changing the prior in 'Gaussian Mixture'

- Using a multimodal prior that matches the number of modes of the true distribution is not realistic;
- The multimodality of the posterior landscape - typically unknown - is precisely one of the main sources of difficulty in complex sampling procedures, especially in applications involving real data.
- To clarify what is expected to be observed in the fish-tracking inference, the following configurations were studied:
 - a. unimodal prior: $\mu_1, \mu_2 \sim \mathcal{N}([0, 0]^\top, 5.0I)$
 - b. bimodal prior, modes further from real values (already tested)
 - c. bimodal prior, modes close to real values: $\mu_1 \sim \mathcal{N}([-4.5, -4.5]^\top, 5.0I)$
 $\mu_2 \sim \mathcal{N}([4.5, 4.5]^\top, 5.0I)$

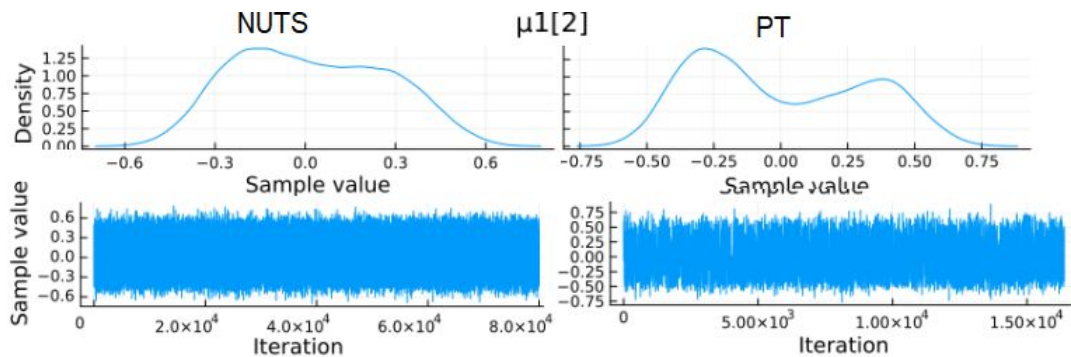
RESULTS - Changing the prior in 'GM' - Scenario a.

4 chains - 14 rounds

- Estimates are totally off, for both samplers (RWMH was not used, to save resources):



- While the “traces” seem to show no pathological behavior, from the densities it is evident that the algorithms are not working properly:



scans	restarts	Λ
2	0	2
...		
4096	61	2.77
8192	109	2.76
16384	211	2.77

high, despite the poor estimates, because all the chains are stuck near 0, so the associated distributions look similar → swaps often accepted

RESULTS - Changing the prior in 'GM' - Scenario a.

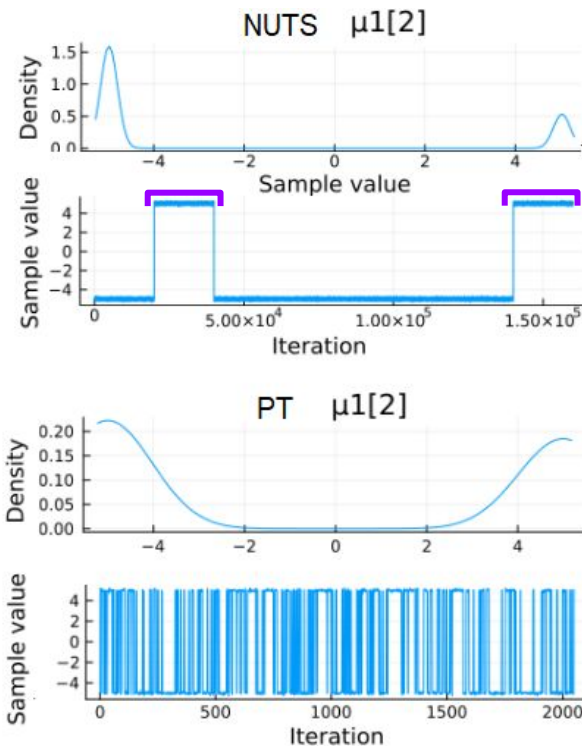
($\geq$)8 chains - 12 rounds

- The situation improves for both samplers but differently between the two:
- NUTS: it recovers the behavior of scenario b., meaning it eventually handles the more complex scenario, but only by chance and improperly, since symmetric bimodality should be present here.
- **NUTS inability to properly explore the posterior is thus further assessed.**
- PT: behaves exactly as it should. Since the problem is symmetric, the algorithm should not prioritize any mode, and so it does, efficiently traversing the space that separates the two modes.

scans	restarts	λ
2	0	2
...		
2048	30	5.5
4096	65	5.5
8192	152	5.5

- **PT stability and accuracy is thus further assessed.**

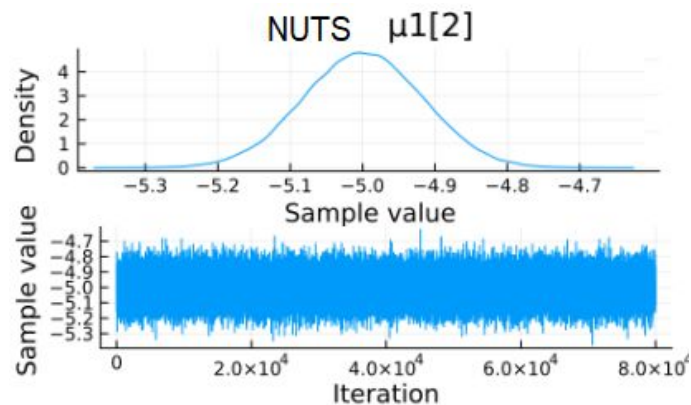
the chains are still initialized around zero, and the higher n_chains smooths the landscape → proper exploration → proper estimates too.



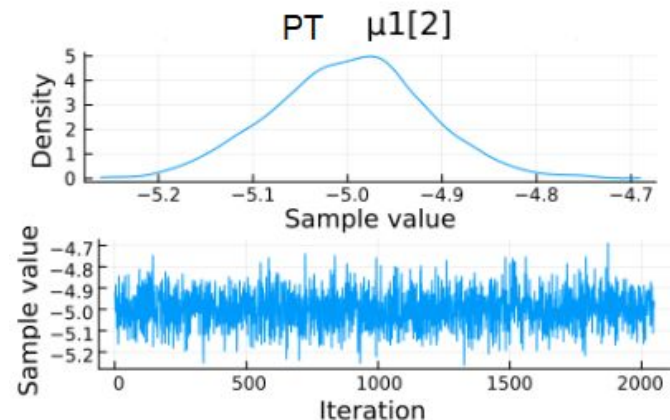
RESULTS - Changing the prior in 'GM' - Scenario c.

4 chains - 11 rounds

- This scenario confirms that a proper initialization allows NUTS to provide proper estimates, which, however, can not always be provided, especially when distributions are complex and unknown.



- Already with 4 chains and 11 rounds only, also PT is able to extract good estimates, and because the posterior is asymmetric (only through the prior though), it detects which mode is the correct one to sample from.



scans	restarts	λ
2	0	2
...		
2048	16	2.8
4096	34	2.8
8192	77	2.8

in scenario b. they were fewer because, apparently, the prior was too far away from the true modes $\rightarrow$ accepting swaps, thus becoming similar to the prior, is not favorable. Here instead the prior is closer to the true distribution $\rightarrow$ swaps can be accepted

RESULTS - Changing the prior in 'GM' - General prescriptions

- It has been shown that the number of restarts must be significantly different from zero, unless the prior is particularly informative about which region is best to sample from; however, this is an information that is rarely available.
- Conversely, if the information provided by the prior is only partial, then a low number of restarts implies poor exploration of the posterior (essentially standard MCMC on the cold chain, and therefore not true PT) → risk of getting trapped in a local minimum/maximum.
- A high number of restarts, however, does not by itself guarantee good estimates if the prior is very far from the posterior; an adequate number of chains is also required.

RESULTS - explorers comparison

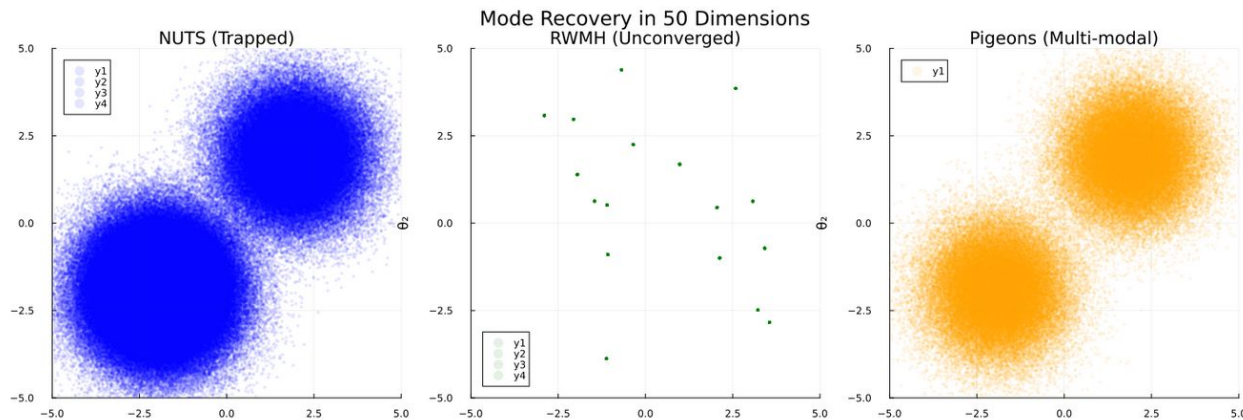
SliceSampler VS AutoMALA

Row	Model	Sampler	Mean_ESS_per_time	Mean_ESS	Mean_R_hat
	String	String	Float64	Float64	Float64
1	Donut	Pigeons_Slice	1605.74	264.751	0.99688
2	Squiggle	Pigeons_Slice	654.571	207.02	0.998809
3	Banana	Pigeons_Slice	114.838	214.257	0.999165
4	Neals_Funnel	Pigeons_Slice	74.0913	247.688	1.00092
5	Gaussian_Mixture	Pigeons_Slice	39.7636	240.826	0.997312
6	Squiggle	Pigeons_AutoMALA	2.13835	247.523	1.00148
7	Donut	Pigeons_AutoMALA	1.83852	163.464	1.00685
8	Banana	Pigeons_AutoMALA	1.08847	181.67	1.0092
9	Gaussian_Mixture	Pigeons_AutoMALA	0.787554	261.908	0.996677
10	Neals_Funnel	Pigeons_AutoMALA	0.674693	217.009	1.00194

- Across all tested distributions, **Slice Sampler** is the most efficient explorer.
- While both samplers achieve convergence, the Slice Sampler is **significantly faster** because it is not gradient-based, avoiding the heavy computational cost.

RESULTS - scaling in high dimension

- Using a Gaussian mixture with the modes in $\{-2, 2\}$ in each dimension, the ability of recovering the distribution from dim50 to dim2 is here studied:
- RWMH fails to explore the 50 dimension space
- NUTS fails to explore all modes and becomes trapped
- PT enables global exploration by swapping between tempered chains.



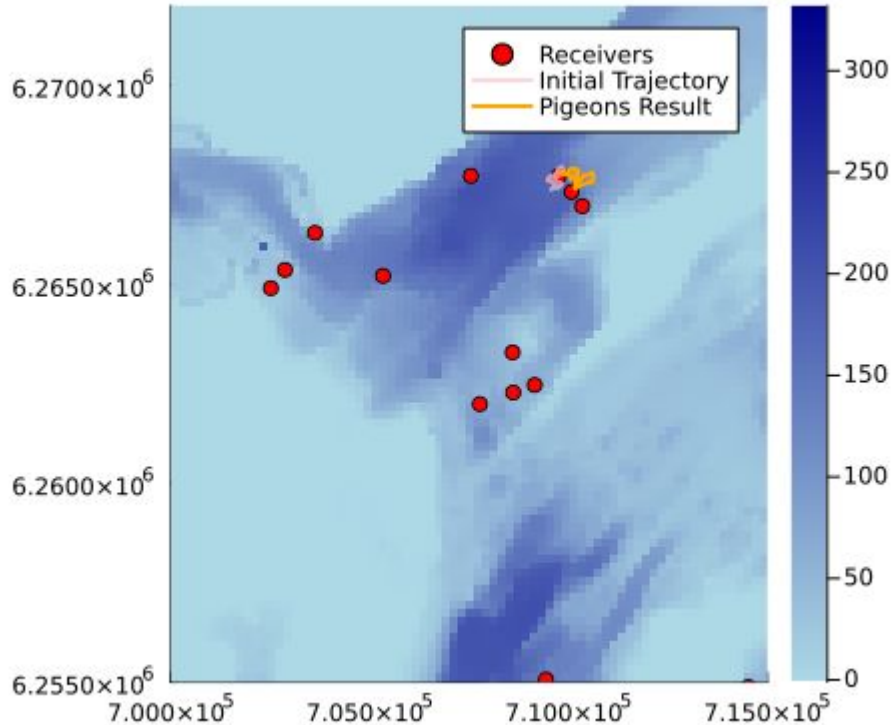
Row	Model	Sampler	Mean_ESS_per_time	Mean_R_hat	Mean_ESS
	String	String	Float64	Float64	Float64
1	D2	RWMH	3265.17	1.0001	36937.7
2	D2	NUTS	332.158	1.00031	10171.8
3	D50	Pigeons	142.33	1.00099	142.33
4	D2	Pigeons	179.822	1.00236	179.822
5	D10	Pigeons	165.933	1.00275	165.933
6	D20	Pigeons	124.199	1.01476	124.199
7	D20	NUTS	4.62426	1.51825	1852.81
8	D50	NUTS	9.28467	1.51826	1852.97
9	D10	RWMH	84.0922	1.56828	1063.32
10	D50	RWMH	45.4608	1.63839	1061.08
11	D10	NUTS	21.6533	1.71892	1585.59
12	D20	RWMH	80.0302	2.22831	1058.29

Fish Tracking



Fish tracking

Comparison of trajectories



- Bathymetry map
- receivers location
- Acoustic data every 2 minutes
- Depth data every 2 minutes
- 1 month of data

s0 is the only known point

Fish tracking

$$p(S \mid Y_{\text{depth}}, Y_{\text{acoustic}}, \text{bathymetry}) \propto p(Y_{\text{depth}} \mid S, \text{bathymetry}) p(Y_{\text{acoustic}} \mid S) p(S)$$

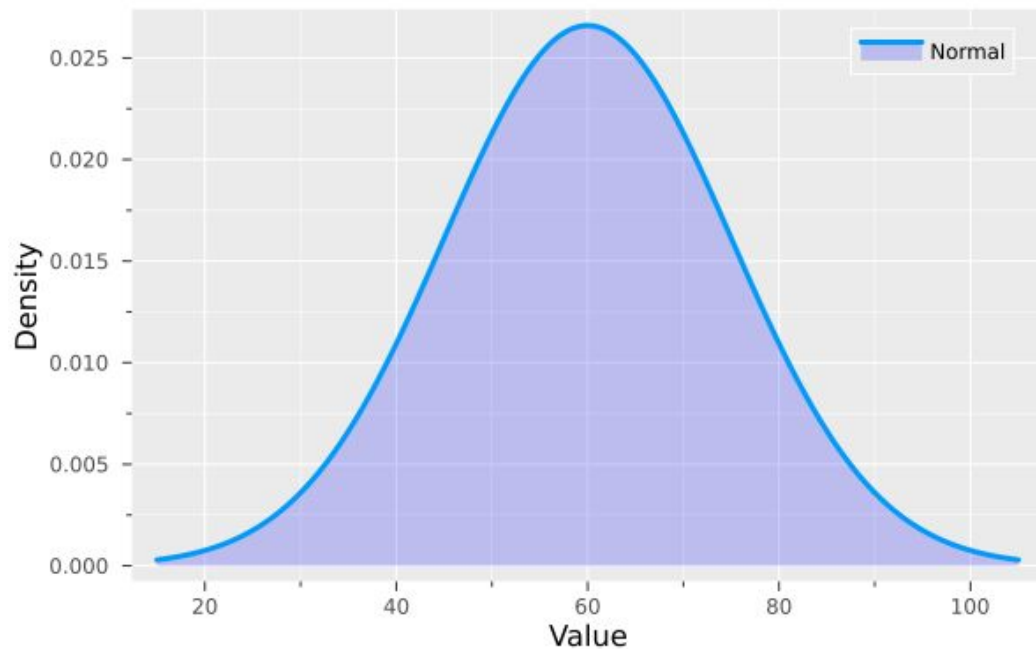
(200 + 1 + 1)D probabilistic vector

- 100 pairs (x,y)
- σ_{depth}
- σ_{move}

Fish tracking - Prior distribution

$$\log p(\mathbf{X}) = \sum_{t=2}^T [\log \mathcal{N}(x_t | x_{t-1}, \sigma) + \log \mathcal{N}(y_t | y_{t-1}, \sigma)]$$

Prior σ : Normal(60, 15)



Fish tracking - Likelihoods

$$p(\text{:detect} \mid X_t, r) = 1 - \frac{1}{1 + \exp(-(d - d_0)/k)}, \quad p(\text{:nondetect} \mid X_t, r) = 1 - p(\text{:detect})$$

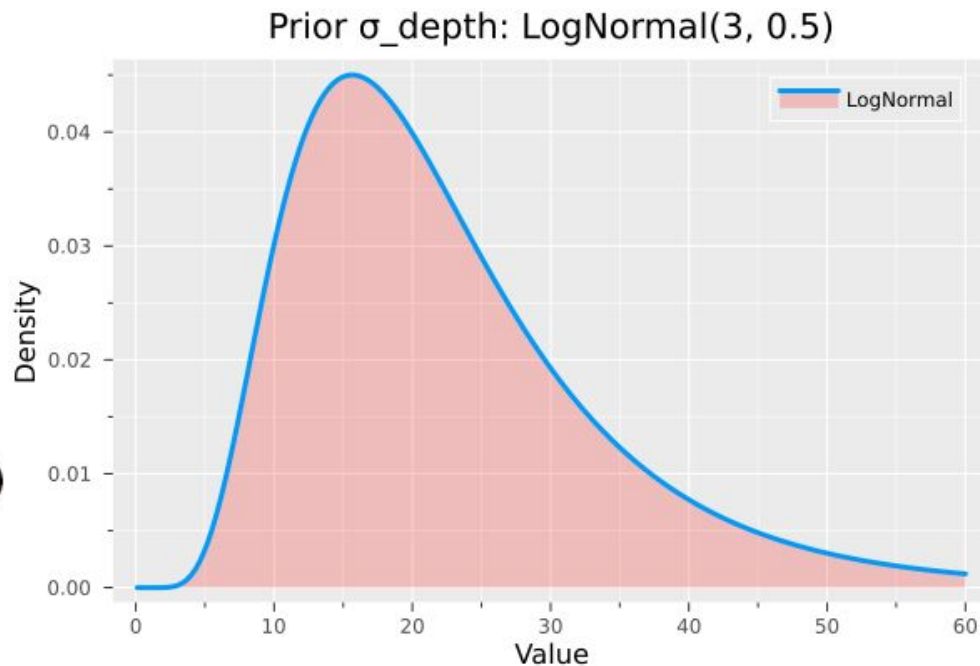
$$\log p(\mathbf{Y}_a \mid \mathbf{X}) = \sum_{(t,s,r) \in \mathbf{Y}_a} \log p(s \mid X_t, r)$$

Fish tracking - Likelihoods

$$p(z | X_t) = \mathcal{N}(z | z_{max}, \sigma)$$

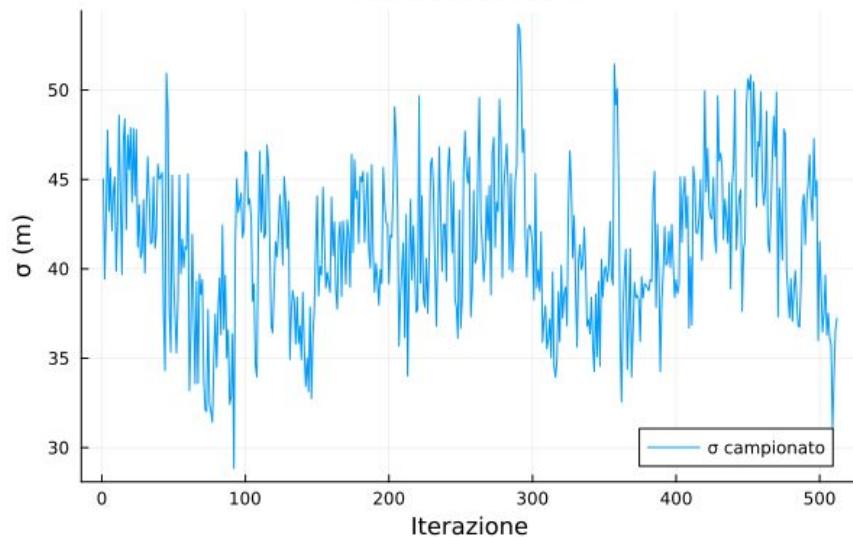
σ : uncertainty on the depth

$$\log p(\mathbf{Y_d} | \mathbf{X}) = \sum_{(t,z) \in \mathbf{Y_d}} \log(\mathcal{N}(z_{max}, \sigma), z)$$

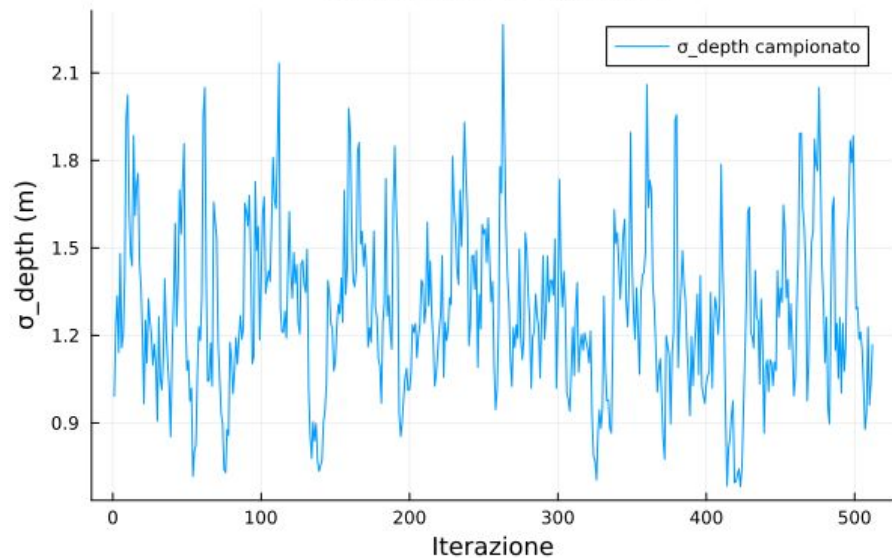


Fish Tracking - RW Results

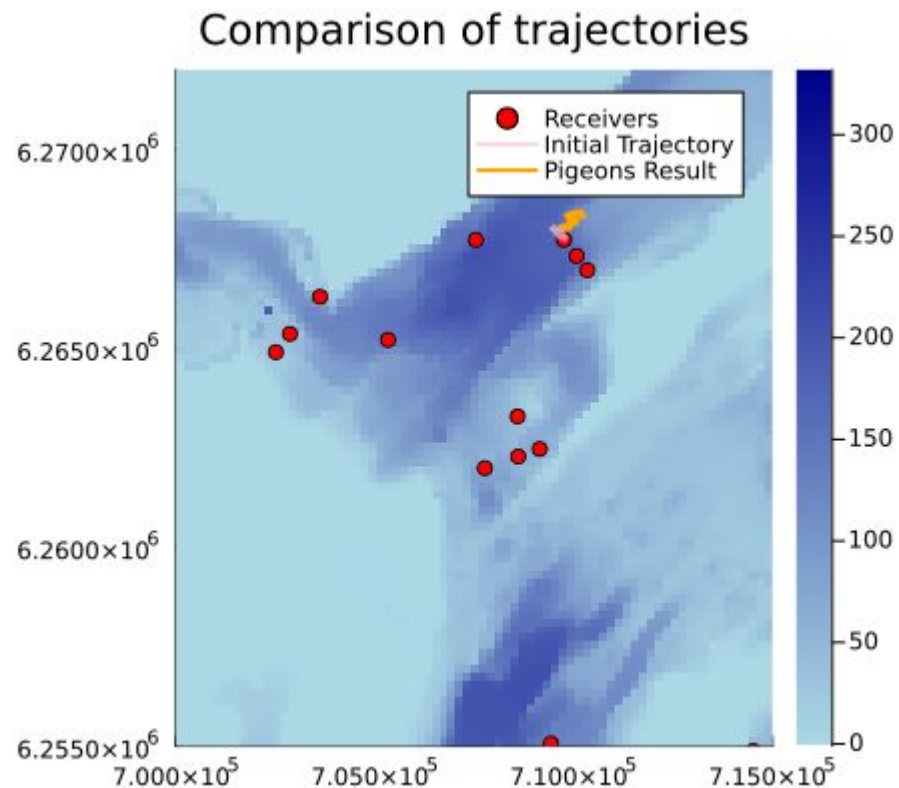
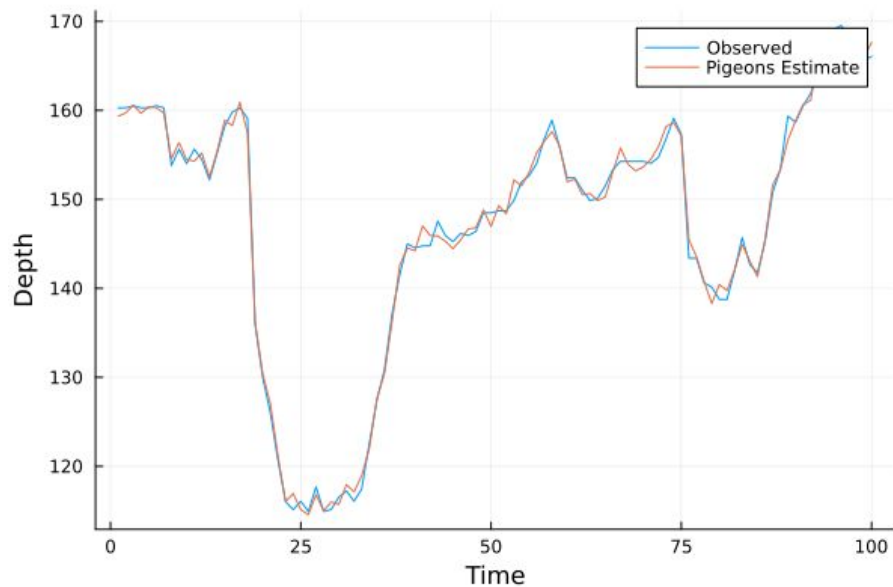
Evoluzione di σ



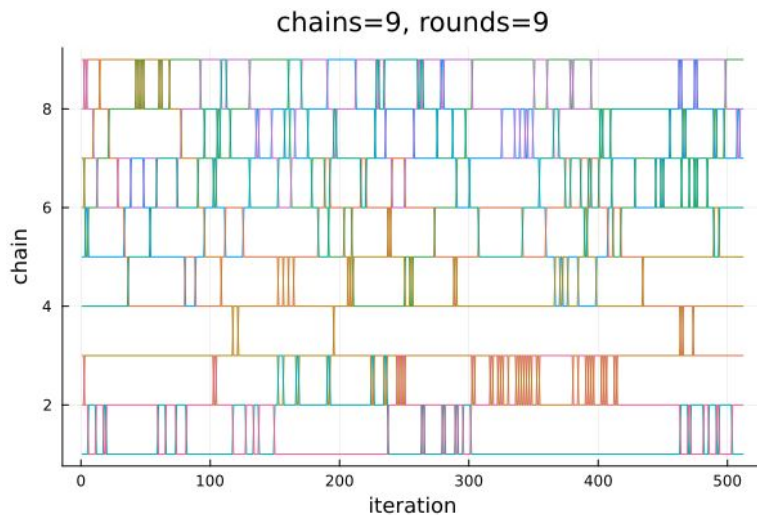
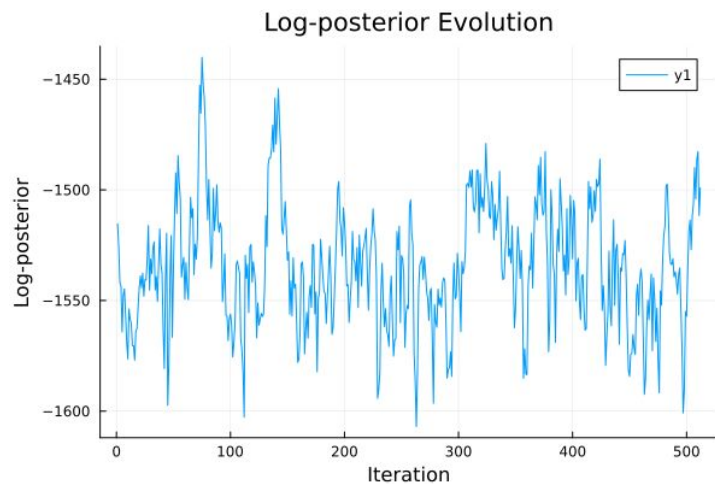
Evoluzione di σ_{depth}



Fish Tracking - RW Results

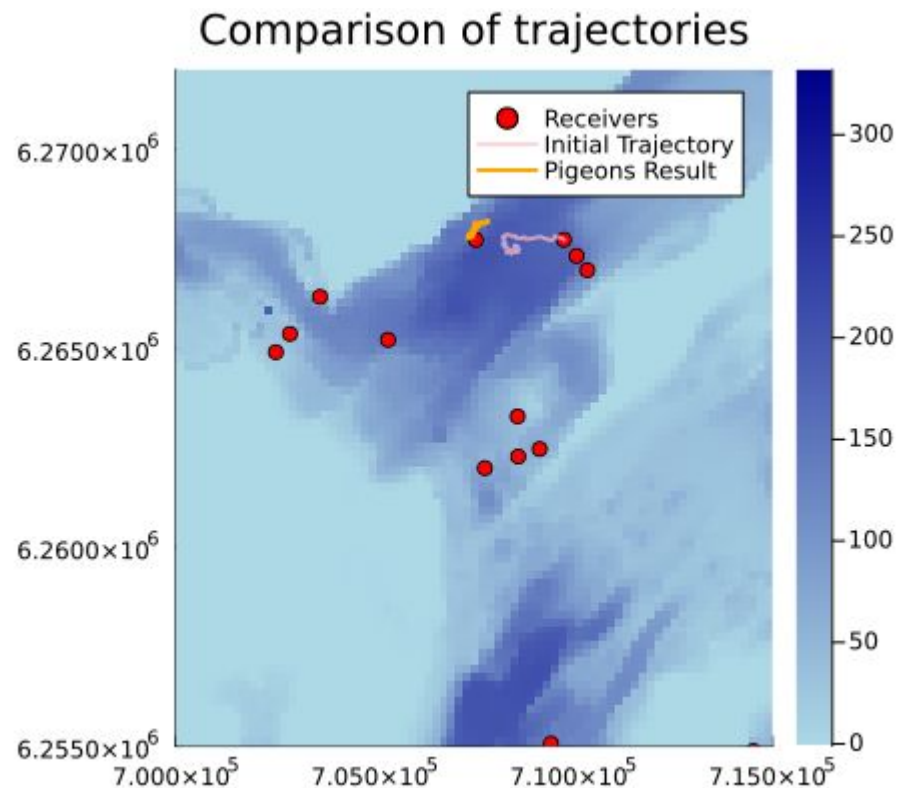
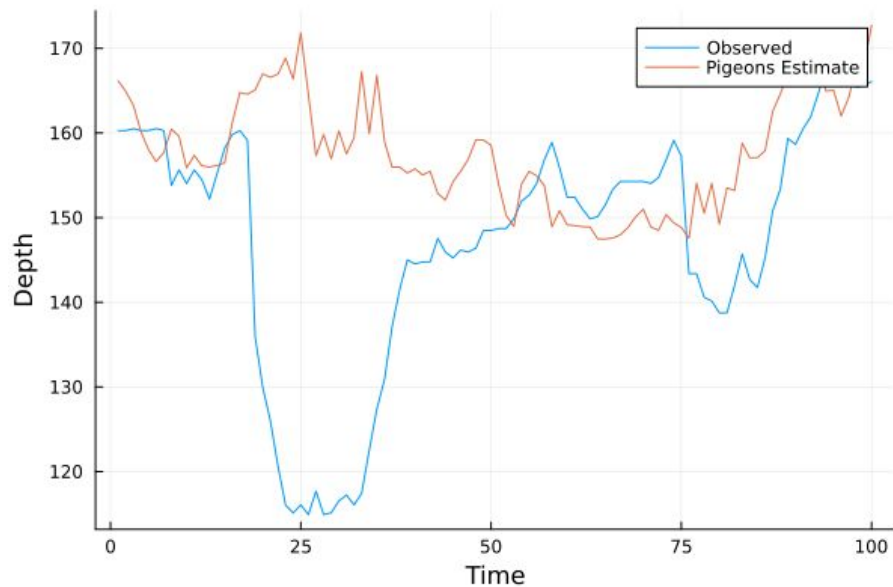


Fish Tracking - RW Results



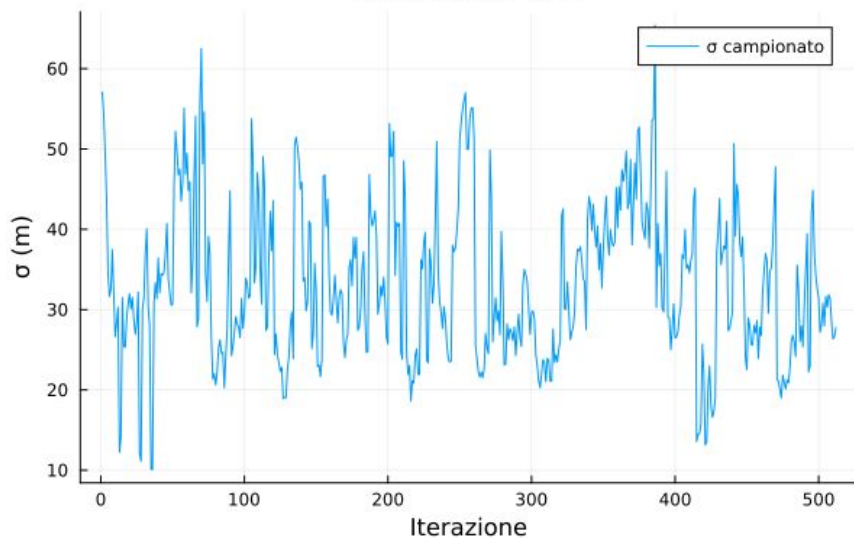
scans	restarts	Λ	time(s)	allc(B)	$\log(Z_1/Z_0)$	$\min(\alpha)$	$\text{mean}(\alpha)$	$\min(\alpha_e)$	$\text{mean}(\alpha_e)$
2	0	1.02	19.5	2.34e+09	-829	0.273	0.873	1	1
4	0	1.99	35.4	4.79e+09	-830	0.189	0.751	1	1
8	0	1.42	72.6	9.58e+09	-822	0.266	0.823	1	1
16	0	2.82	157	1.96e+10	-786	0.199	0.647	1	1
32	0	6.13	296	4.03e+10	-709	0.0638	0.233	1	1
64	0	6.62	579	7.9e+10	-640	4.18e-05	0.173	1	1
128	0	7.2	1.22e+03	1.58e+11	-636	0.00511	0.0999	1	1
256	0	7.15	2.48e+03	3.16e+11	-637	0.0104	0.106	1	1
512	0	7.06	4.67e+03	6.28e+11	-638	0.0221	0.117	1	1

Fish Tracking - BRW Results

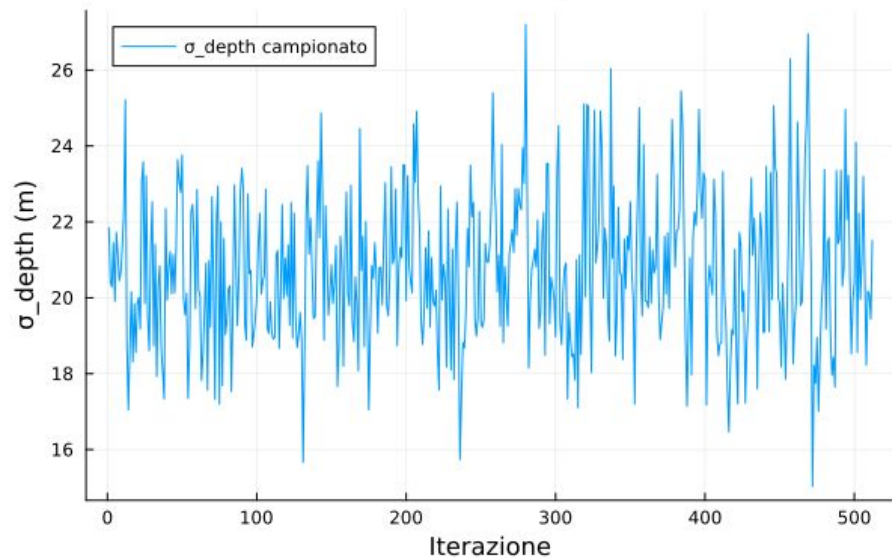


Fish Tracking - BRW Results

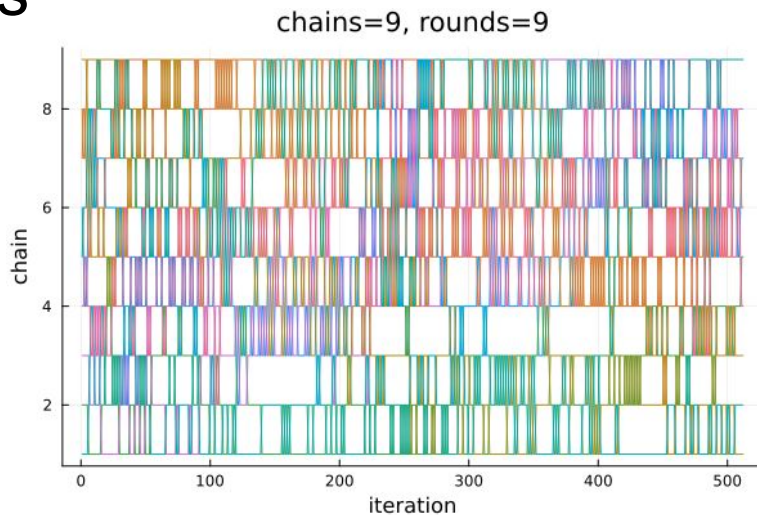
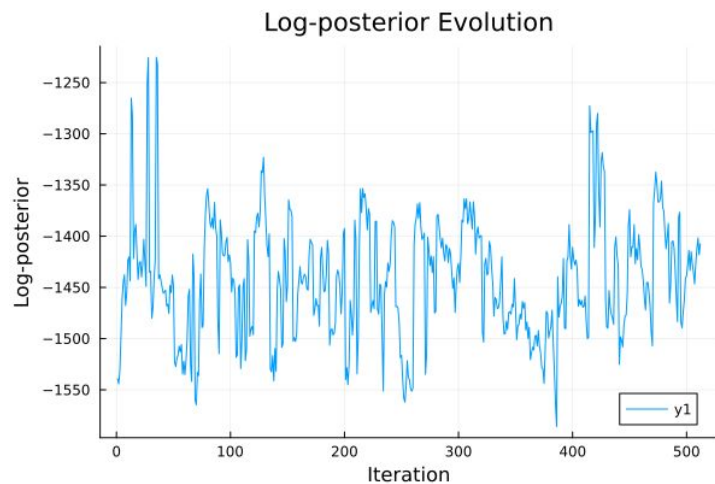
Evoluzione di σ



Evoluzione di σ_{depth}



Fish Tracking - BRW Results



scans	restarts	Λ	time(s)	allc(B)	$\log(Z_1/Z_0)$	$\min(\alpha)$	$\text{mean}(\alpha)$	$\min(\alpha_e)$	$\text{mean}(\alpha_e)$
2	0	1.13	19.2	2.43e+09	-780	0.55	0.859	1	1
4	0	2.57	40.1	5.24e+09	-685	0.0925	0.678	1	1
8	0	3.19	83.8	1.13e+10	-570	0.251	0.602	1	1
16	0	4.33	162	2.26e+10	-545	0.141	0.459	1	1
32	0	5.18	315	4.36e+10	-532	0.148	0.353	1	1
64	0	4.94	611	8.45e+10	-511	0.0438	0.382	1	1
128	1	4.68	1.18e+03	1.65e+11	-498	0.168	0.414	1	1
256	0	4.46	2.26e+03	3.15e+11	-494	0.123	0.442	1	1
512	1	4.41	4.42e+03	6.17e+11	-492	0.337	0.449	1	1